

Solving Problems by Searching

Russell & Norvig - AIMA

Ioan Alfred Letia

Outline

- Problem-solving agents
- Example problems: toy problems, real-world problems
- Searching for solutions
- Uninformed search strategies: breadth-first, depth-first, depth-limited, iterative deepening, bidirectional
- Avoiding repeated states
- Searching with partial information
- Summary

Problem-Solving Agents

- static, observable, discrete, deterministic environment

function SIMPLE-PROBLEM-SOLVING-AGENT(*percept*) **returns** an action

static: *seq*, an action sequence, initially empty

state, some description of the current world state

goal, a goal, initially null

problem, a problem formulation

state $\leftarrow$ UPDATE-STATE(*state*, *percept*)

if *seq* is empty **then**

goal $\leftarrow$ FORMULATE-GOAL(*state*)

problem $\leftarrow$ FORMULATE-PROBLEM(*state*, *goal*)

seq $\leftarrow$ SEARCH(*problem*)

action $\leftarrow$ RECOMMENDATION(*seq*, *state*)

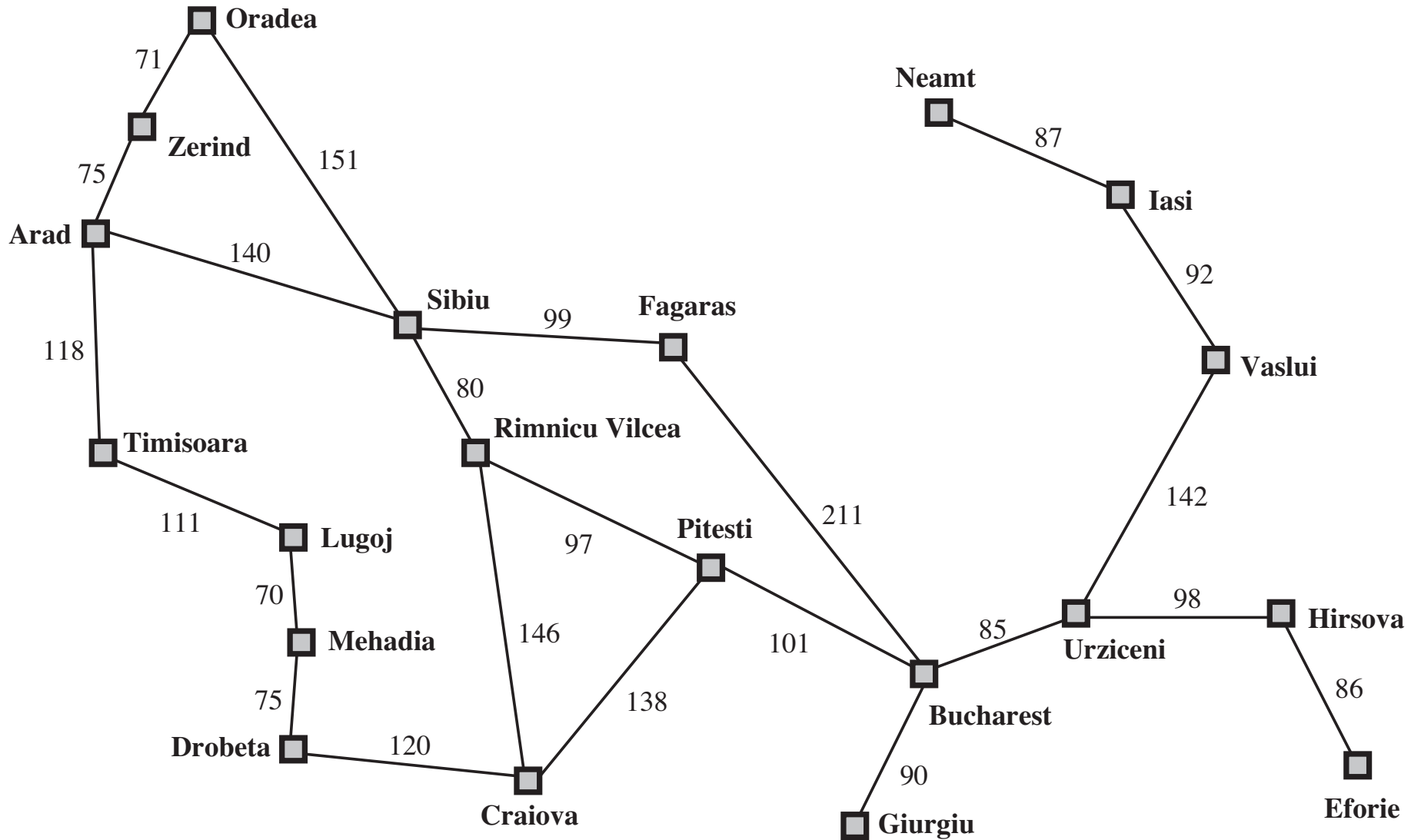
seq $\leftarrow$ REMAINDER(*seq*, *state*)

return *action*

Example: Romania

- On holiday in Romania; currently in Arad. Flight leaves tomorrow from Bucharest
- Formulate goal: be in Bucharest
- Formulate problem:
 - *states*: various cities
 - *actions*: drive between cities
- Find solution:
 - sequence of cities, e.g., Arad, Sibiu, Fagaras, Bucharest

Example: Romania 2



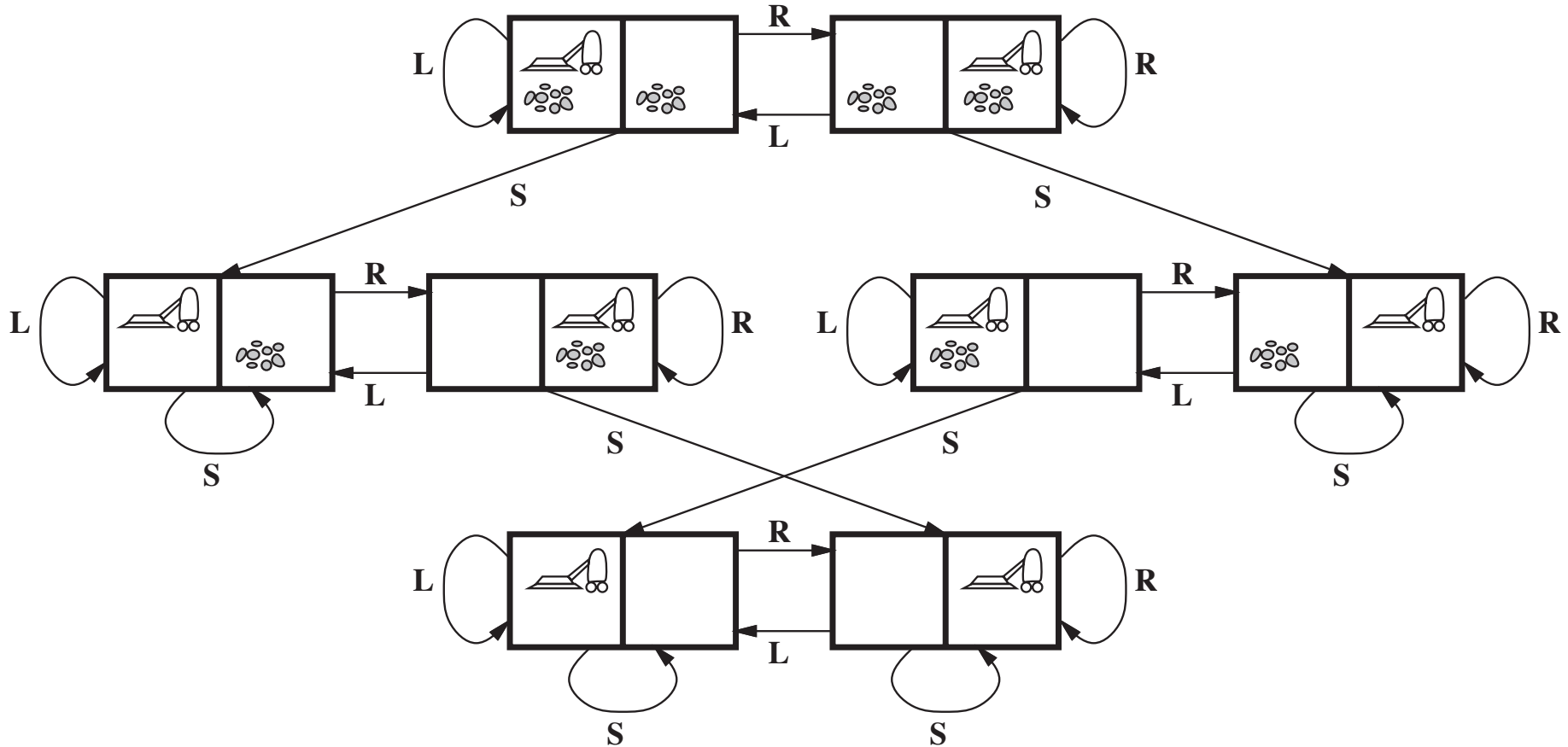
Single-State Problem Formulation

- A *problem* is defined by four items:
 - *initial state* e.g., “at Arad”
 - *successor function* $S(x)$ = set of action–state pairs
e.g., $S(Arad) = \{\langle Arad \rightarrow Zerind, Zerind \rangle, \dots\}$
 - *goal test*, can be
explicit, e.g., $x = \text{“at Bucharest”}$
implicit, e.g., $NoDirt(x)$
 - *path cost* (additive)
e.g., sum of distances, number of actions executed, etc.
 $c(x, a, y)$ is the *step cost*, assumed to be ≥ 0
- A *solution* is a sequence of actions leading from the initial state to a goal state

Selecting a State Space

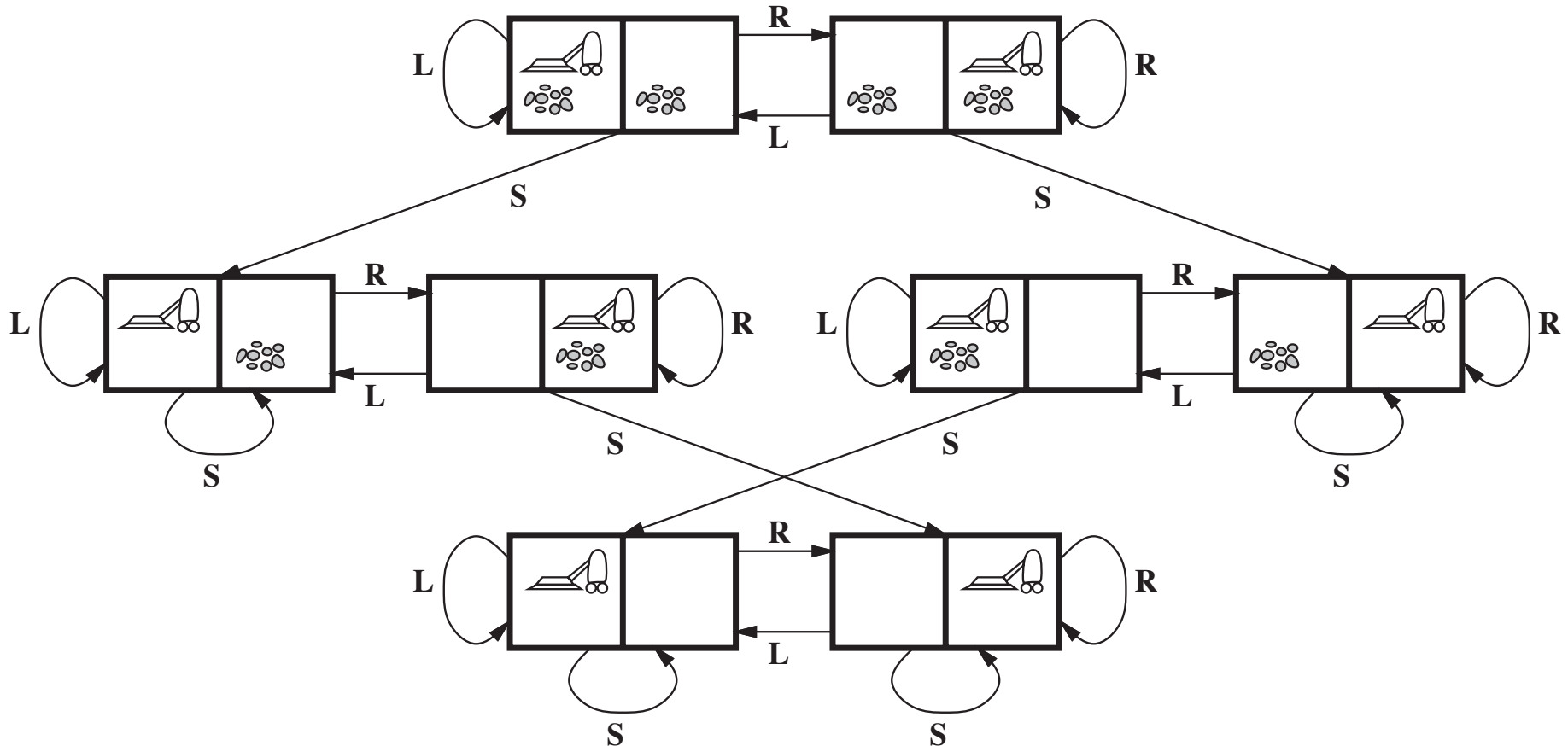
- Real world is absurdly complex
⇒ state space must be *abstracted* for problem solving
- (Abstract) state = set of real states
- (Abstract) action = complex combination of real actions
e.g., “Arad → Zerind” represents a complex set of possible routes, detours, rest stops, etc.
For guaranteed realizability, *any* real state “in Arad” must get to *some* real state “in Zerind”
- (Abstract) solution =
set of real paths that are solutions in the real world
- Each abstract action should be “easier” than the original problem!

Vacuum World State Space Graph



- states??
- actions??
- goal test??
- path cost??

Vacuum World State Space Graph



- states??: integer dirt and robot locations (ignore dirt *amounts*)
- actions??: *Left, Right, Suck, NoOp*
- goal test??: no dirt
- path cost??: 1 per action (0 for *NoOp*)

Example: The 8-Puzzle

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

- states??
- actions??
- goal test??
- path cost??

Example: The 8-Puzzle

7	2	4
5		6
8	3	1

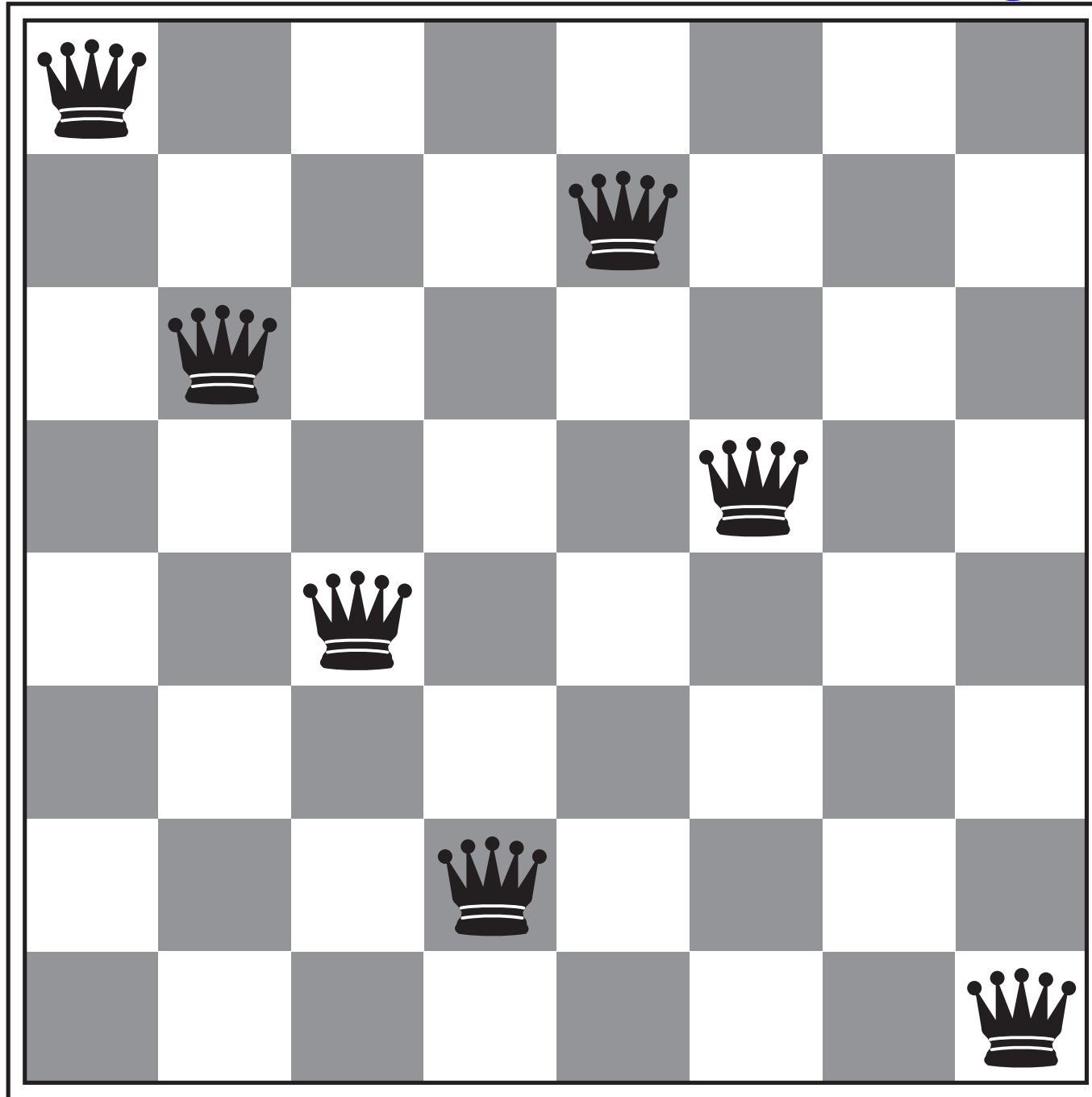
Start State

	1	2
3	4	5
6	7	8

Goal State

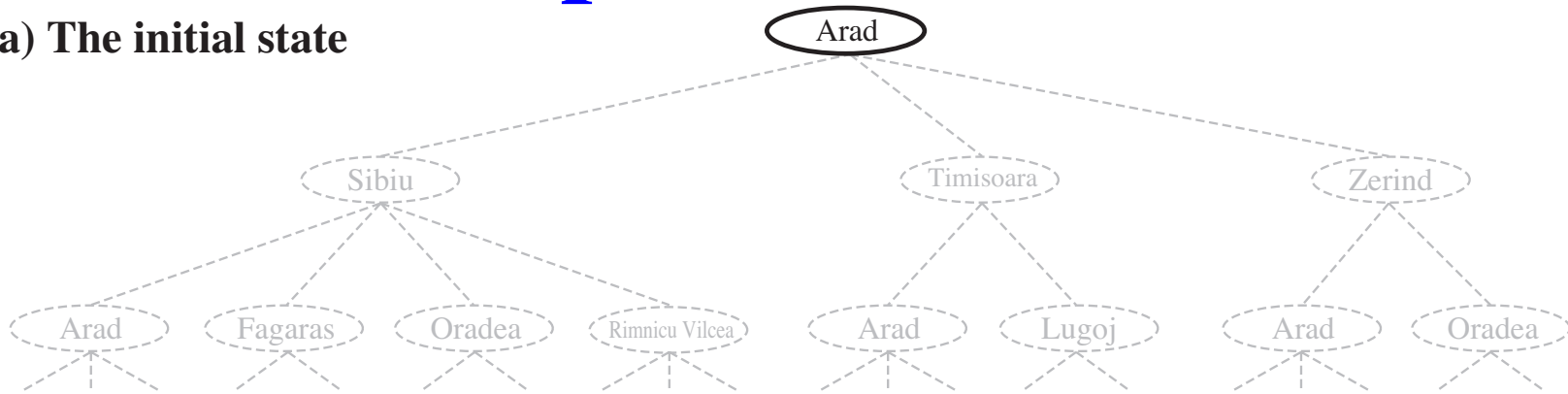
- states??: integer locations of tiles (ignore intermediate positions)
- actions??: move blank left, right, up, down (ignore unjamming etc.)
- goal test??: = goal state (given)
- path cost??: 1 per move

Almost a Solution to 8-Queens

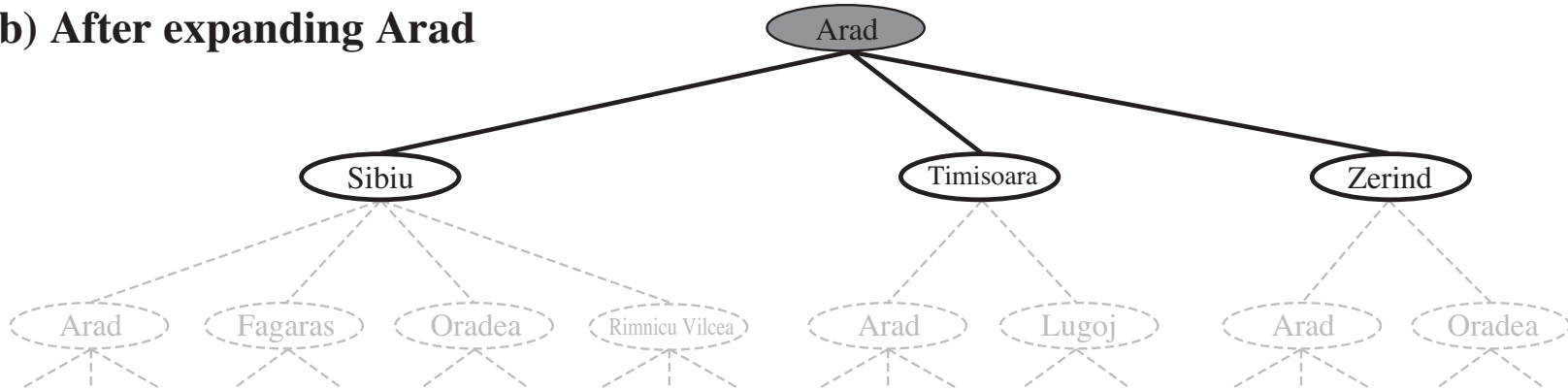


Search Map

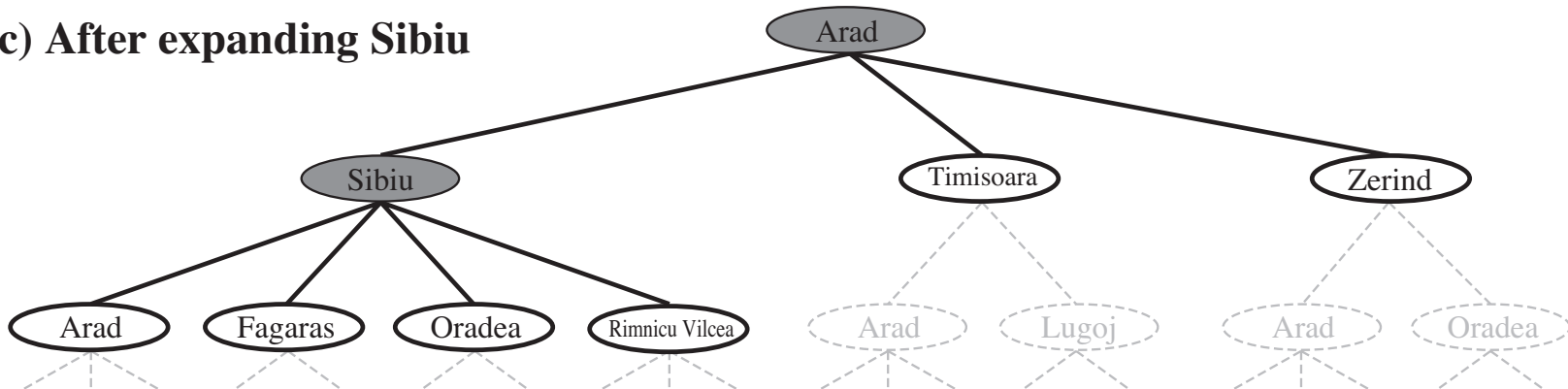
(a) The initial state



(b) After expanding Arad



(c) After expanding Sibiu



Tree Search Algorithms

- Basic idea:
offline, simulated exploration of state space
by generating successors of already-explored states
(a.k.a. *expanding* states)

function TREE-SEARCH(*problem, strategy*) **returns** a solution, or failure

 initialize the search tree using the initial state of *problem*

loop do

 if there are no candidates for expansion **then return** failure

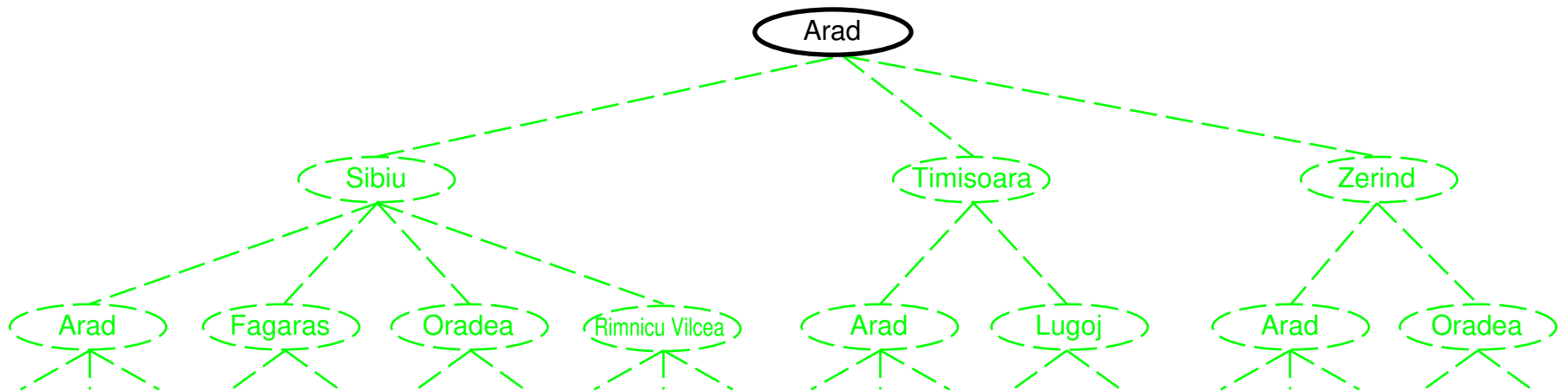
 choose a leaf node for expansion according to *strategy*

 if the node contains a goal state **then return** the corresponding solution

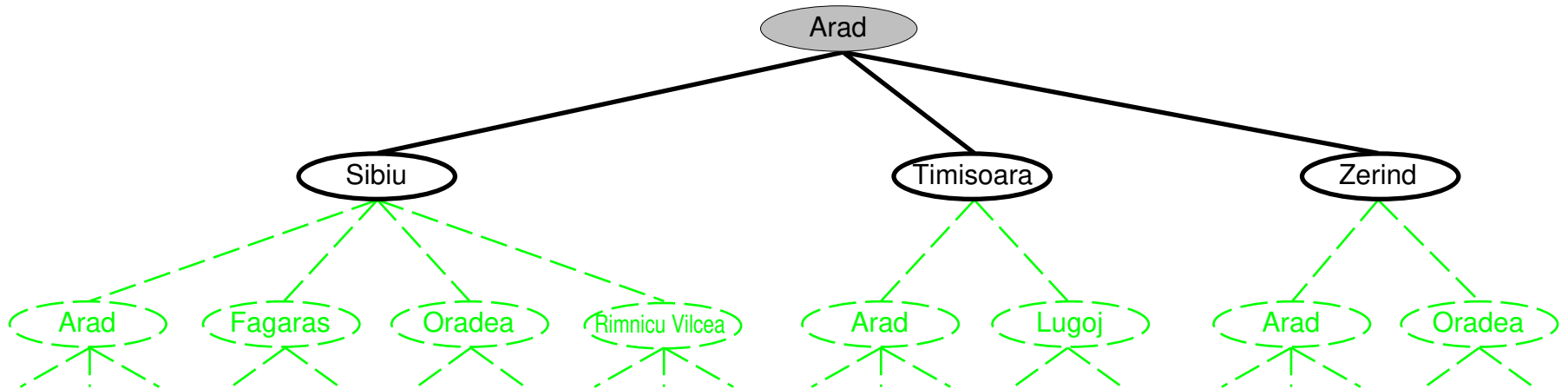
else expand the node and add the resulting nodes to the search tree

end

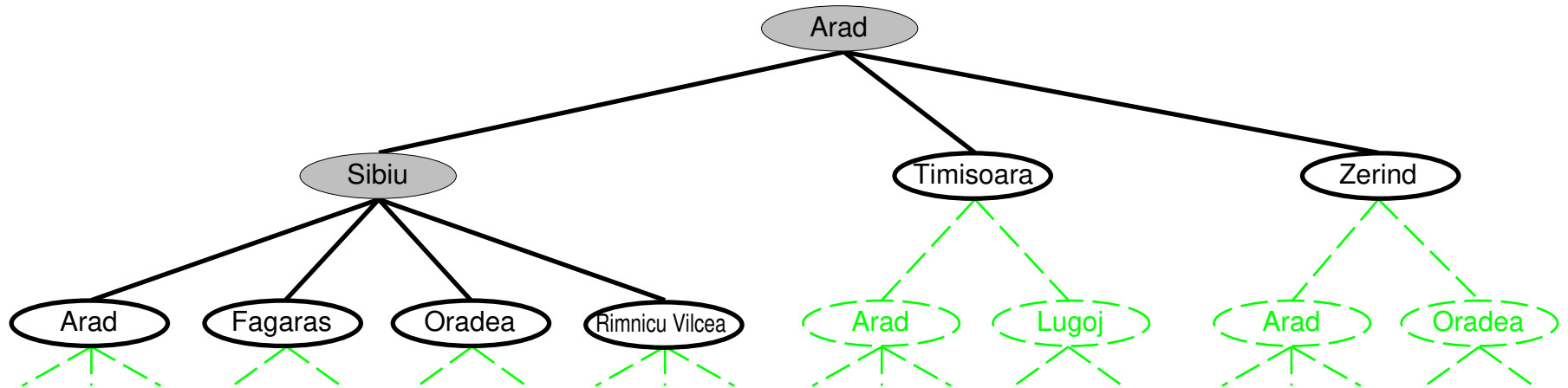
Tree Search Example



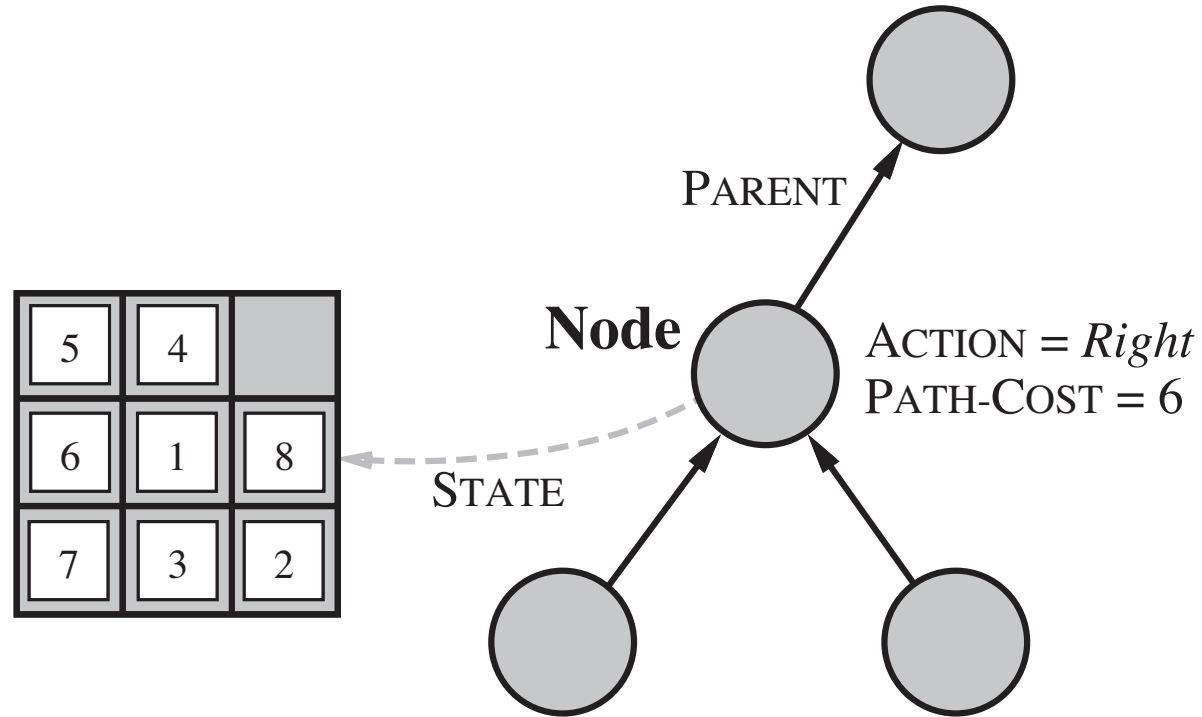
Tree Search Example



Tree Search Example



Implementation: States vs. Nodes



- A *state* is a (representation of) a physical configuration
- A *node* is a data structure constituting part of a search tree includes *parent*, *children*, *depth*, *path cost* $g(x)$
- *States* do not have parents, children, depth, or path cost!
- The EXPAND function creates new nodes, filling in the various fields and using the SUCCESSORFN of the problem to create the corresponding states

General Tree Search

function TREE-SEARCH(*problem*, *fringe*) **returns** a solution, or failure

fringe $\leftarrow$ INSERT(MAKE-NODE(INITIAL-STATE[*problem*]), *fringe*)

loop do

if *fringe* is empty **then return** failure

node $\leftarrow$ REMOVE-FRONT(*fringe*)

if GOAL-TEST(*problem*, STATE(*node*)) **then return** *node*

fringe $\leftarrow$ INSERTALL(EXPAND(*node*, *problem*), *fringe*)

function EXPAND(*node*, *problem*) **returns** a set of nodes

successors $\leftarrow$ the empty set

for each *action*, *result* **in** SUCCESSOR-FN(*problem*, STATE[*node*]) **do**

s $\leftarrow$ a new NODE

 PARENT-NODE[*s*] $\leftarrow$ *node*; ACTION[*s*] $\leftarrow$ *action*; STATE[*s*] $\leftarrow$ *result*

 PATH-COST[*s*] $\leftarrow$ PATH-COST[*node*] + STEP-COST(STATE[*node*], *action*, *result*)

 DEPTH[*s*] $\leftarrow$ DEPTH[*node*] + 1

 add *s* to *successors*

Search Strategies

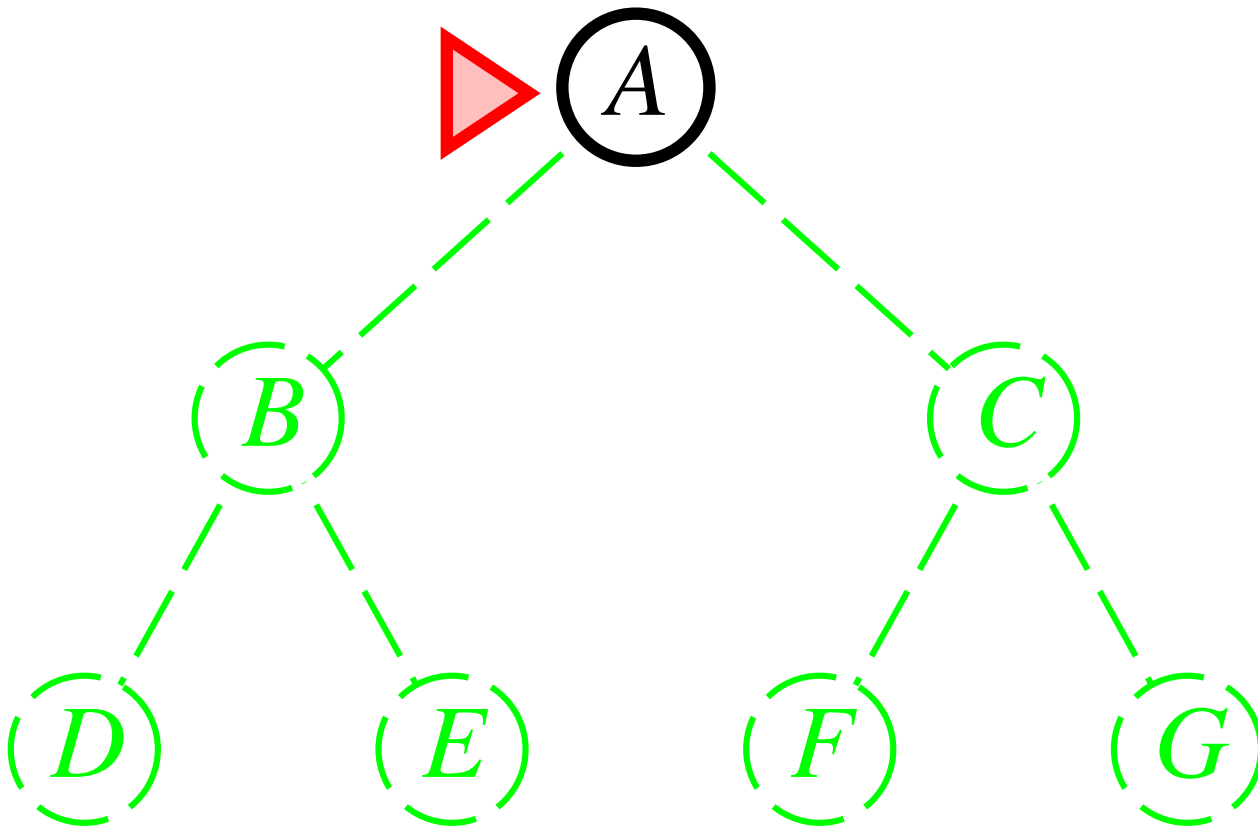
- A strategy is defined by picking the *order of node expansion*
- Strategies are evaluated along the following dimensions:
 - **completeness**—does it always find a solution if one exists?
 - **time complexity**—number of nodes generated/expanded
 - **space complexity**—maximum number of nodes in memory
 - **optimality**—does it always find a least-cost solution?
- Time and space complexity are measured in terms of
 - b —maximum branching factor of the search tree
 - d —depth of the least-cost solution
 - m —maximum depth of the state space (may be ∞)

Uninformed Search Strategies

- *Uninformed* strategies use only the information available in the problem definition
- Breadth-first search
- Uniform-cost search
- Depth-first search
- Depth-limited search
- Iterative deepening search

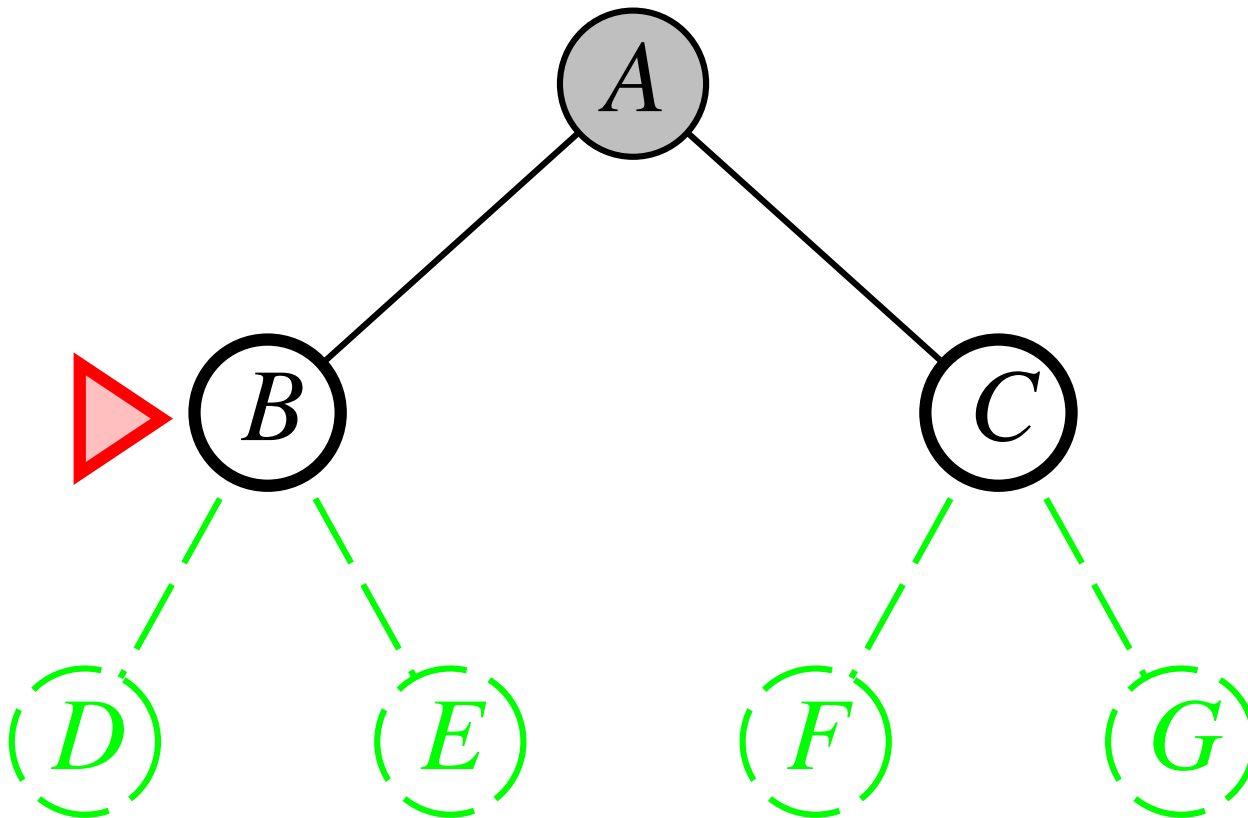
Breadth-First Search

- Expand shallowest unexpanded node
- **Implementation:** *fringe* is a FIFO queue, i.e., new successors go at end



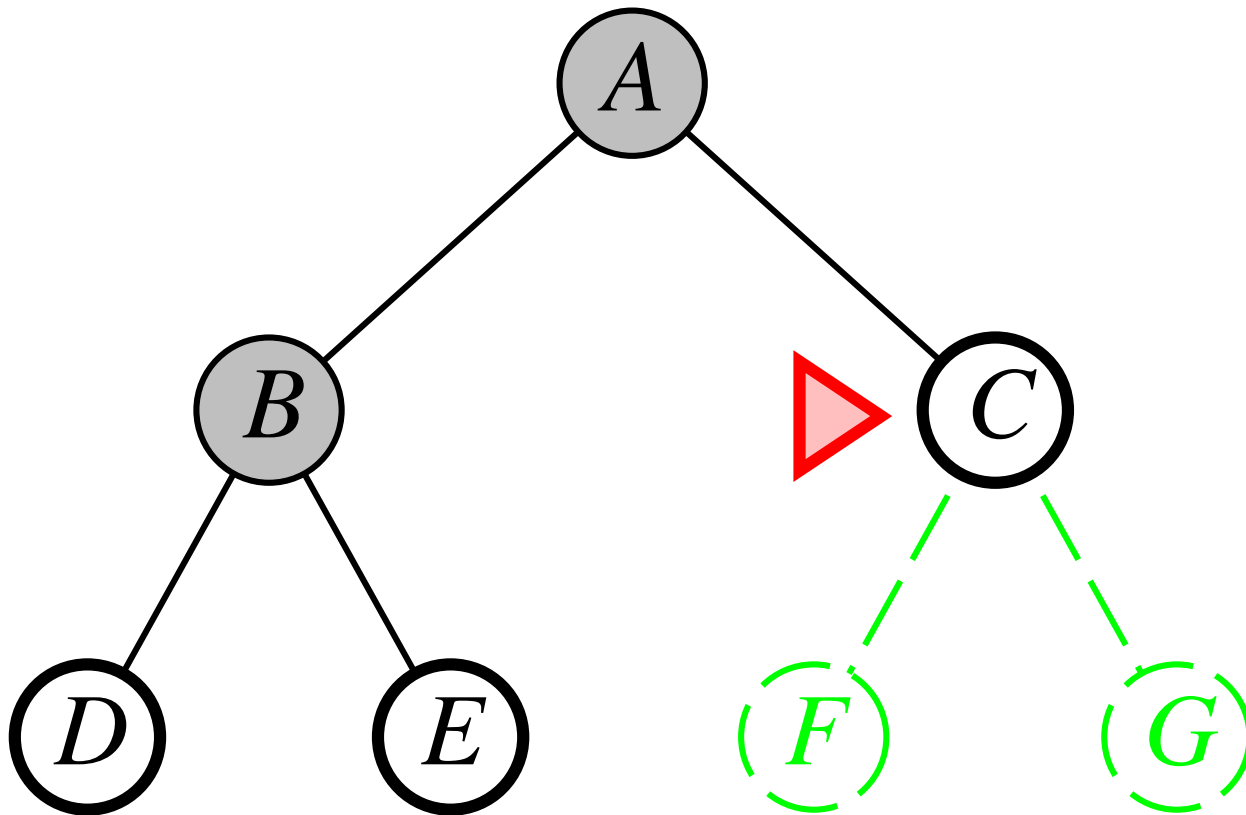
Breadth-First Search

- Expand shallowest unexpanded node
- **Implementation:** *fringe* is a FIFO queue, i.e., new successors go at end



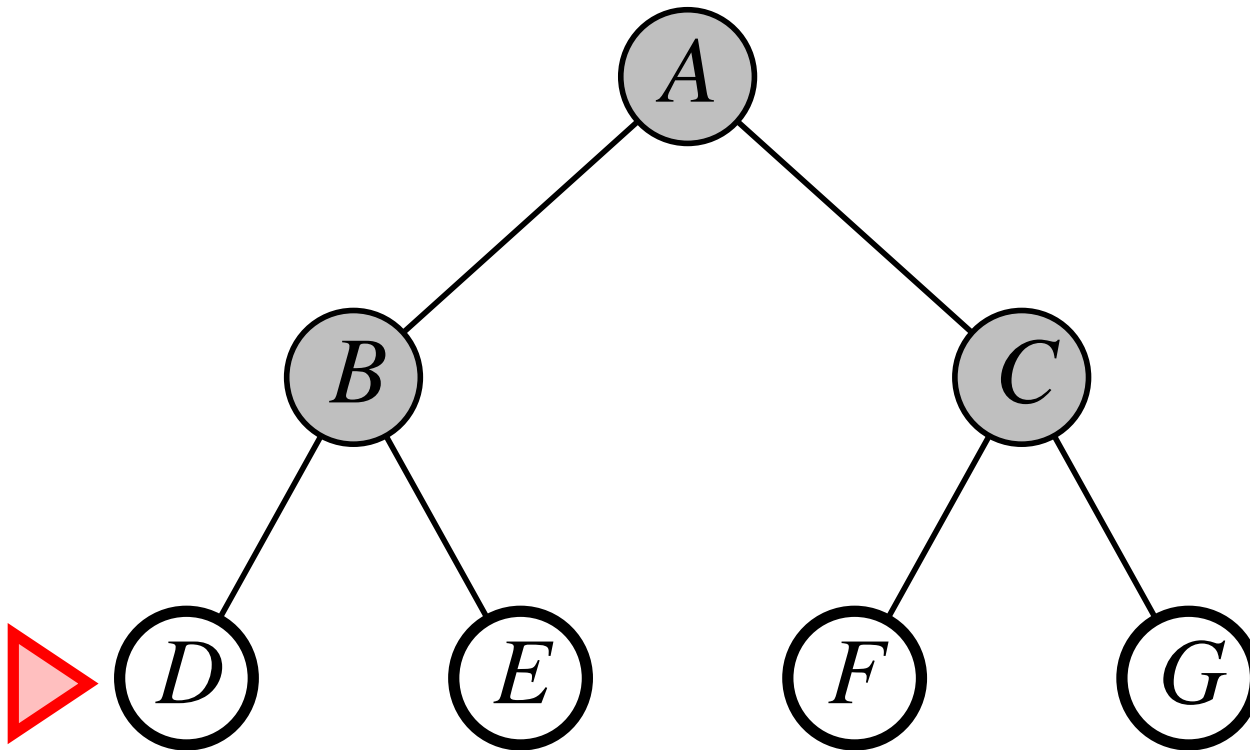
Breadth-First Search

- Expand shallowest unexpanded node
- **Implementation:** *fringe* is a FIFO queue, i.e., new successors go at end

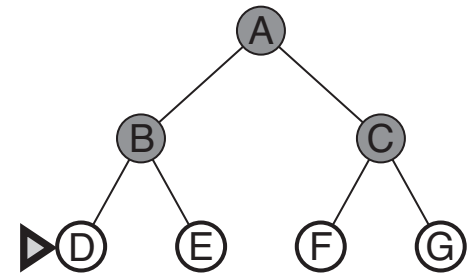
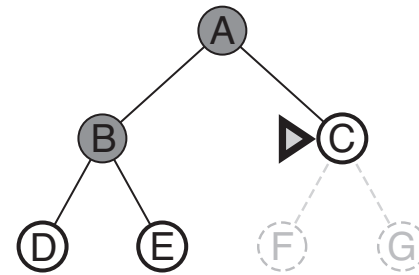
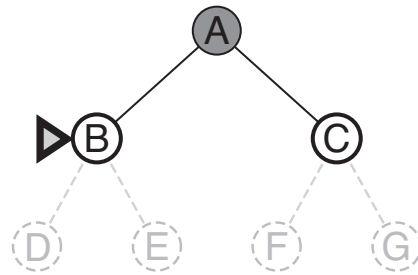
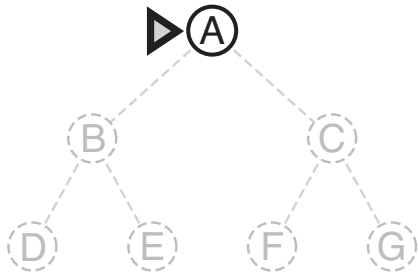


Breadth-First Search

- Expand shallowest unexpanded node
- Implementation:** *fringe* is a FIFO queue, i.e., new successors go at end



BFS Progress



Properties of Breadth-First Search

● Complete??

Properties of Breadth-First Search

- Complete?? Yes (if b is finite)
- Time??

Properties of Breadth-First Search

- Complete?? Yes (if b is finite)
- Time?? $1 + b + b^2 + b^3 + \dots + b^d + b(b^d - 1) = O(b^{d+1})$,
i.e., exp. in d
- Space??

Properties of Breadth-First Search

- Complete?? Yes (if b is finite)
- Time?? $1 + b + b^2 + b^3 + \dots + b^d + b(b^d - 1) = O(b^{d+1})$,
i.e., exp. in d
- Space?? $O(b^{d+1})$ (keeps every node in memory)
- Optimal??

Properties of Breadth-First Search

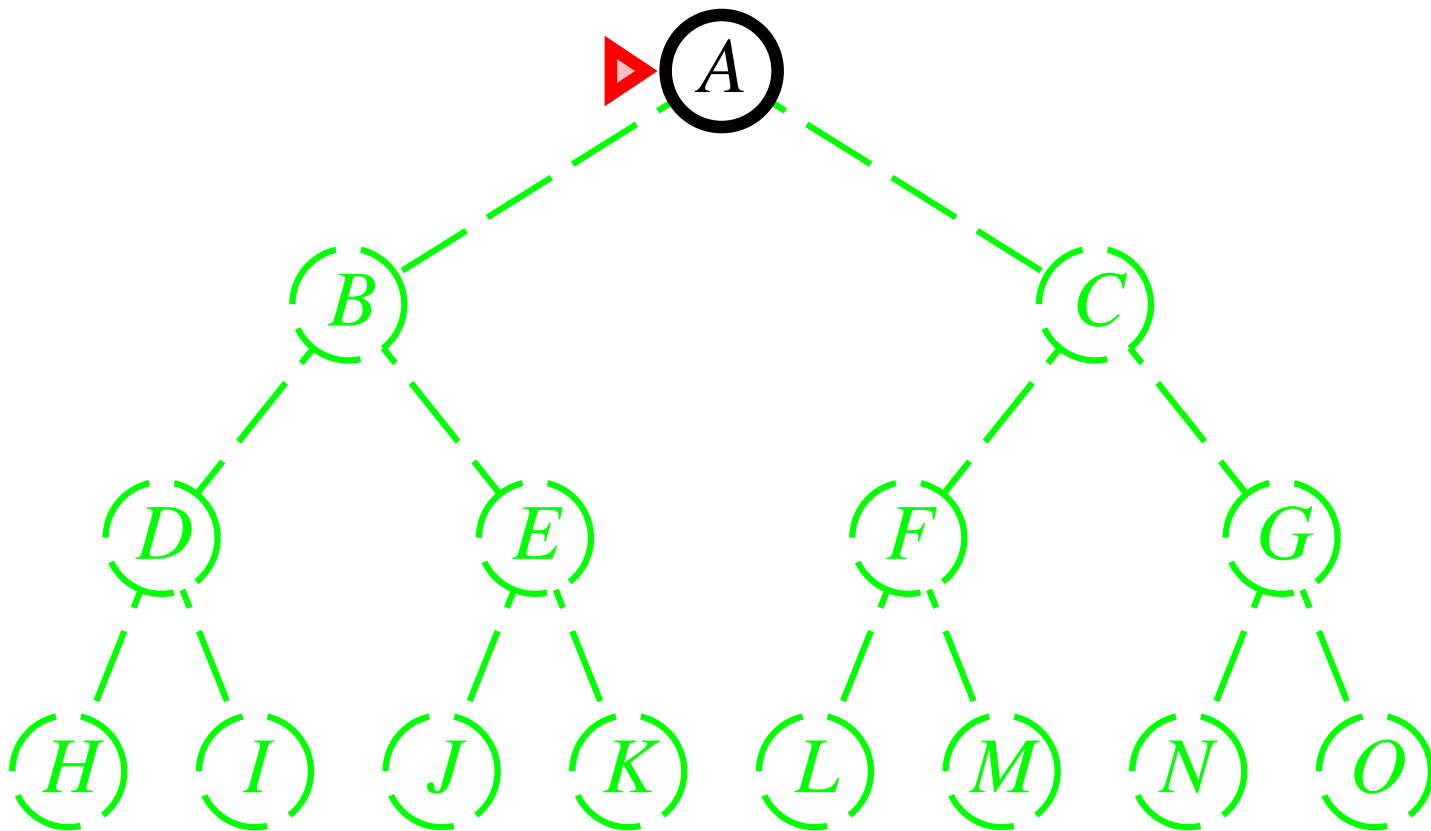
- Complete?? Yes (if b is finite)
- Time?? $1 + b + b^2 + b^3 + \dots + b^d + b(b^d - 1) = O(b^{d+1})$,
i.e., exp. in d
- Space?? $O(b^{d+1})$ (keeps every node in memory)
- Optimal?? Yes (if cost = 1 per step); not optimal in general
- *Space* is the big problem; can easily generate nodes at 10MB/sec so 24hrs = 860GB.

Uninform-Cost Search

- Expand least-cost unexpanded node
- **Implementation:**
fringe = queue ordered by path cost
- Equivalent to breadth-first if step costs all equal
- Complete?? Yes, if step cost $\geq \epsilon$
- Time?? # of nodes with $g \leq$ cost of optimal solution, $O(b^{\lceil C^*/\epsilon \rceil})$, where C^* is the cost of the optimal solution
- Space?? # of nodes with $g \leq$ cost of optimal solution, $O(b^{\lceil C^*/\epsilon \rceil})$
- Optimal?? Yes—nodes expanded in increasing order of $g(n)$

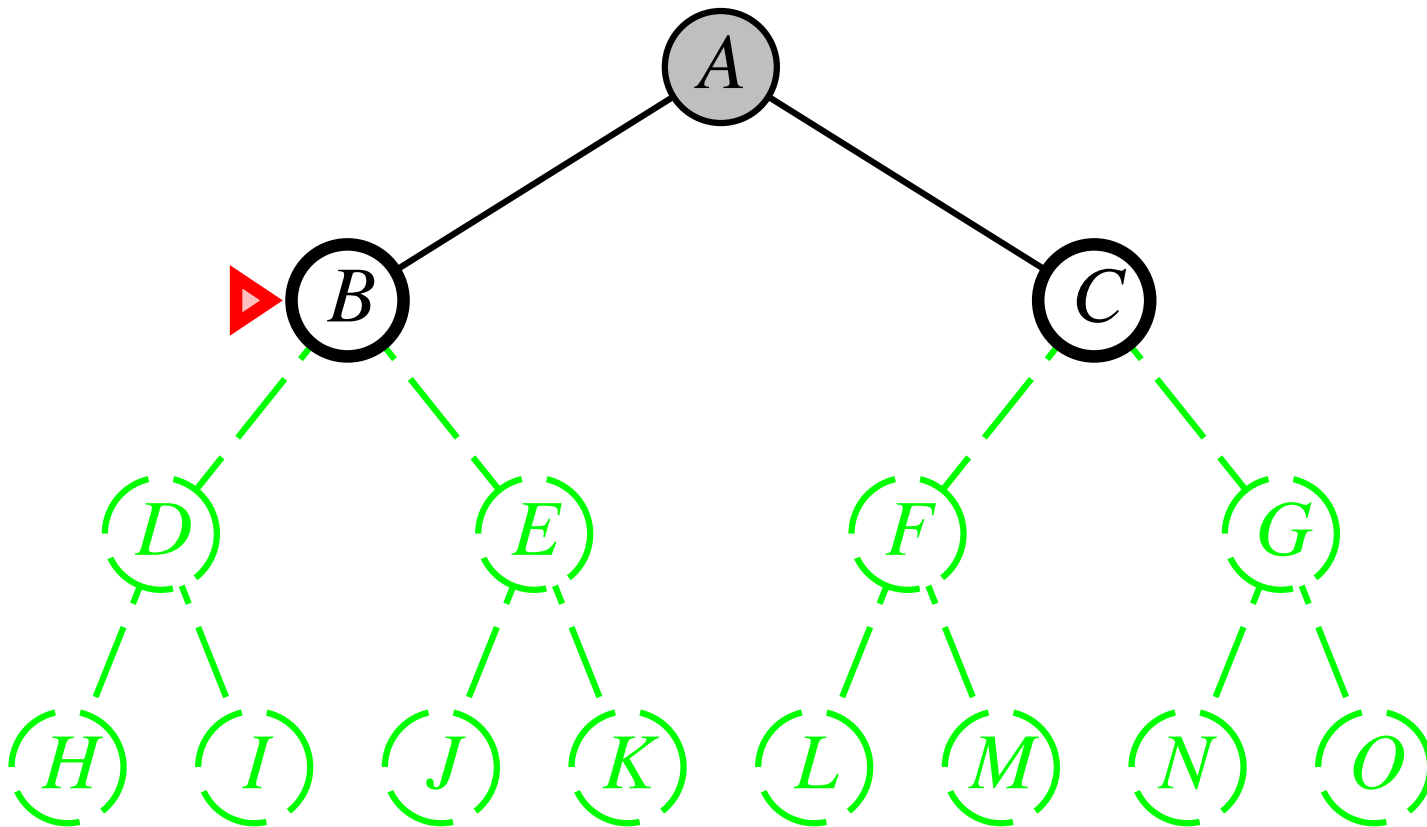
Depth-First Search

- Expand deepest unexpanded node
- **Implementation:** *fringe* = LIFO queue, i.e., put successors at front



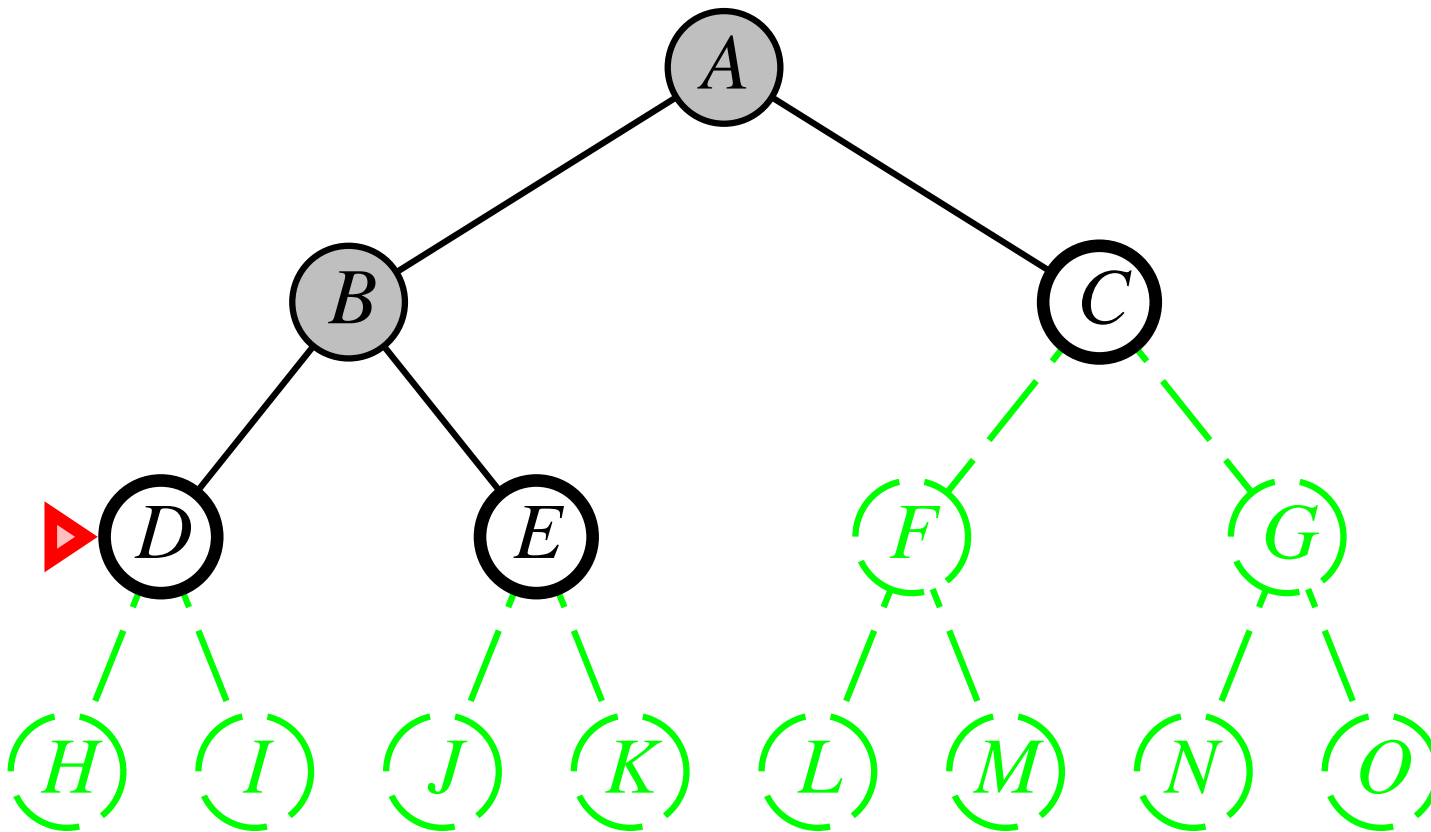
Depth-First Search

- Expand deepest unexpanded node
- Implementation:** *fringe* = LIFO queue, i.e., put successors at front



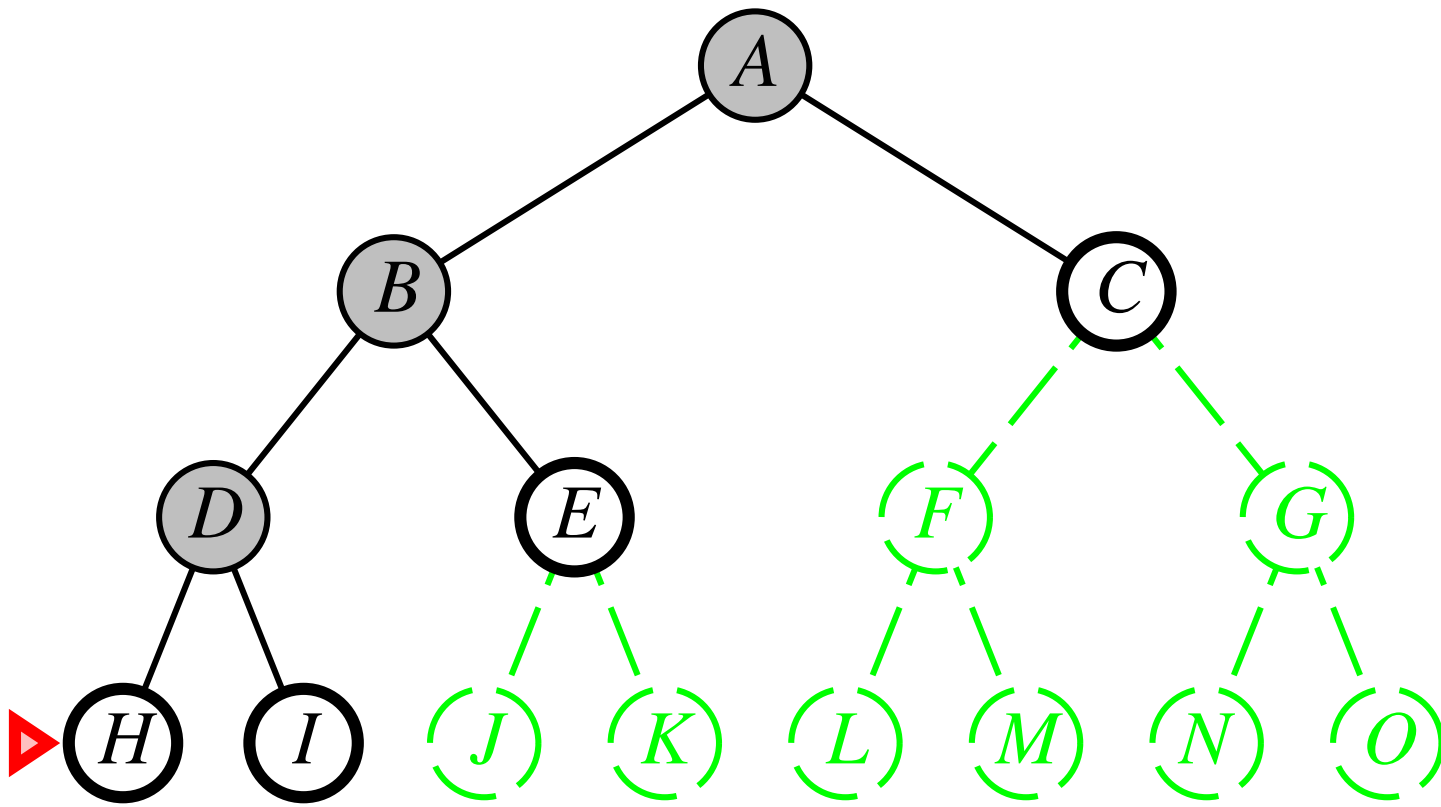
Depth-First Search

- Expand deepest unexpanded node
- Implementation:** *fringe* = LIFO queue, i.e., put successors at front



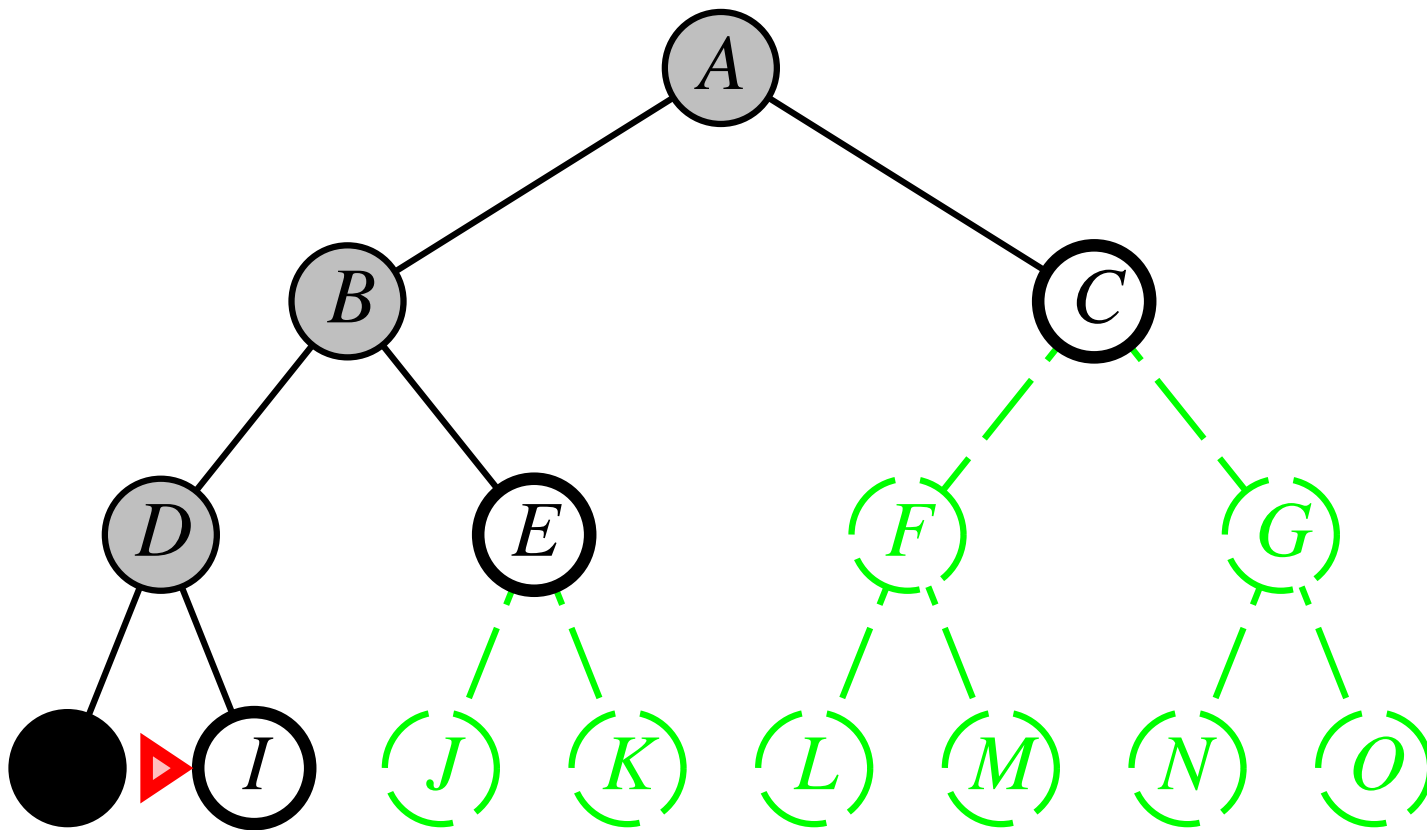
Depth-First Search

- Expand deepest unexpanded node
- Implementation:** *fringe* = LIFO queue, i.e., put successors at front



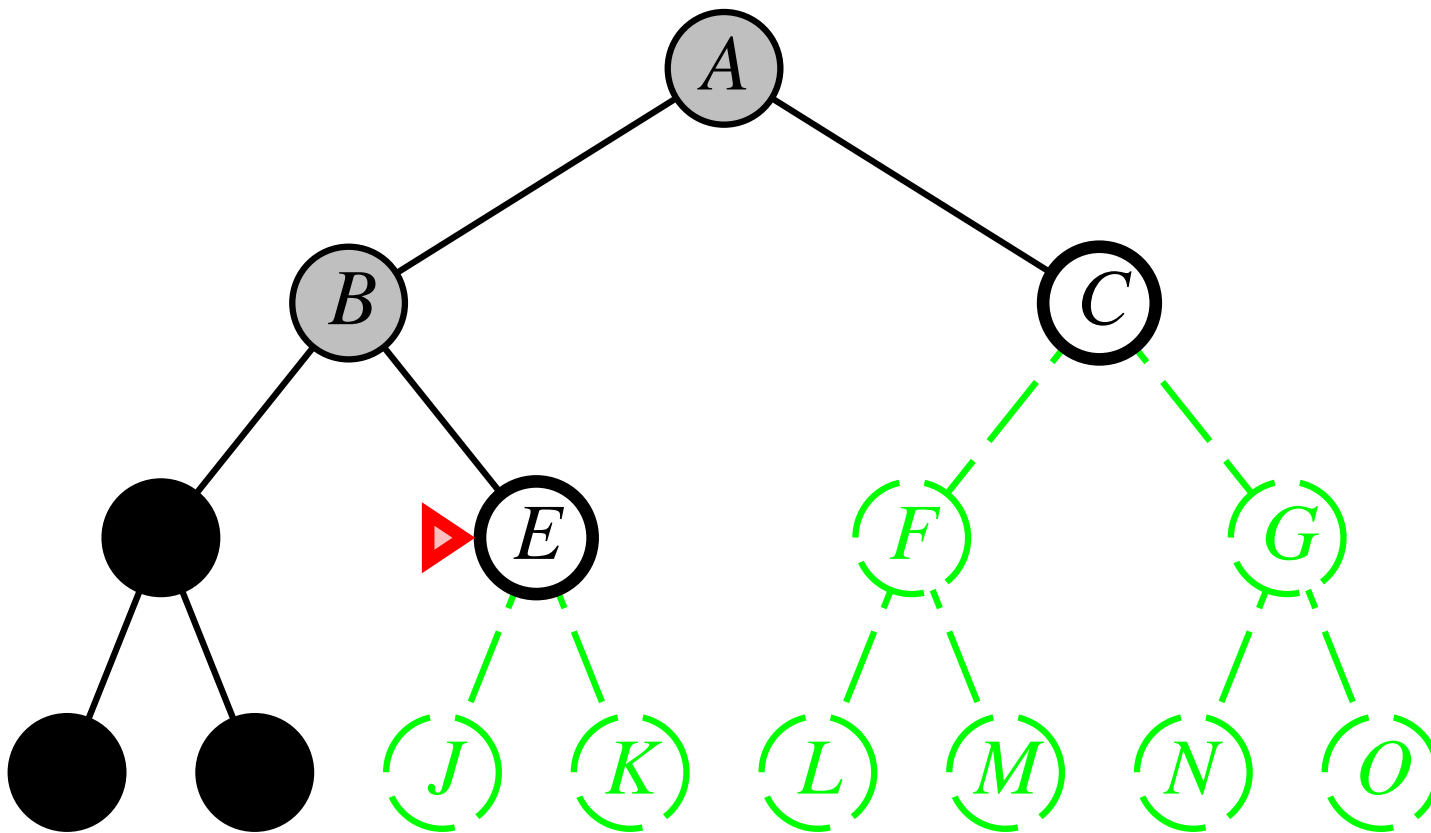
Depth-First Search

- Expand deepest unexpanded node
- Implementation:** *fringe* = LIFO queue, i.e., put successors at front



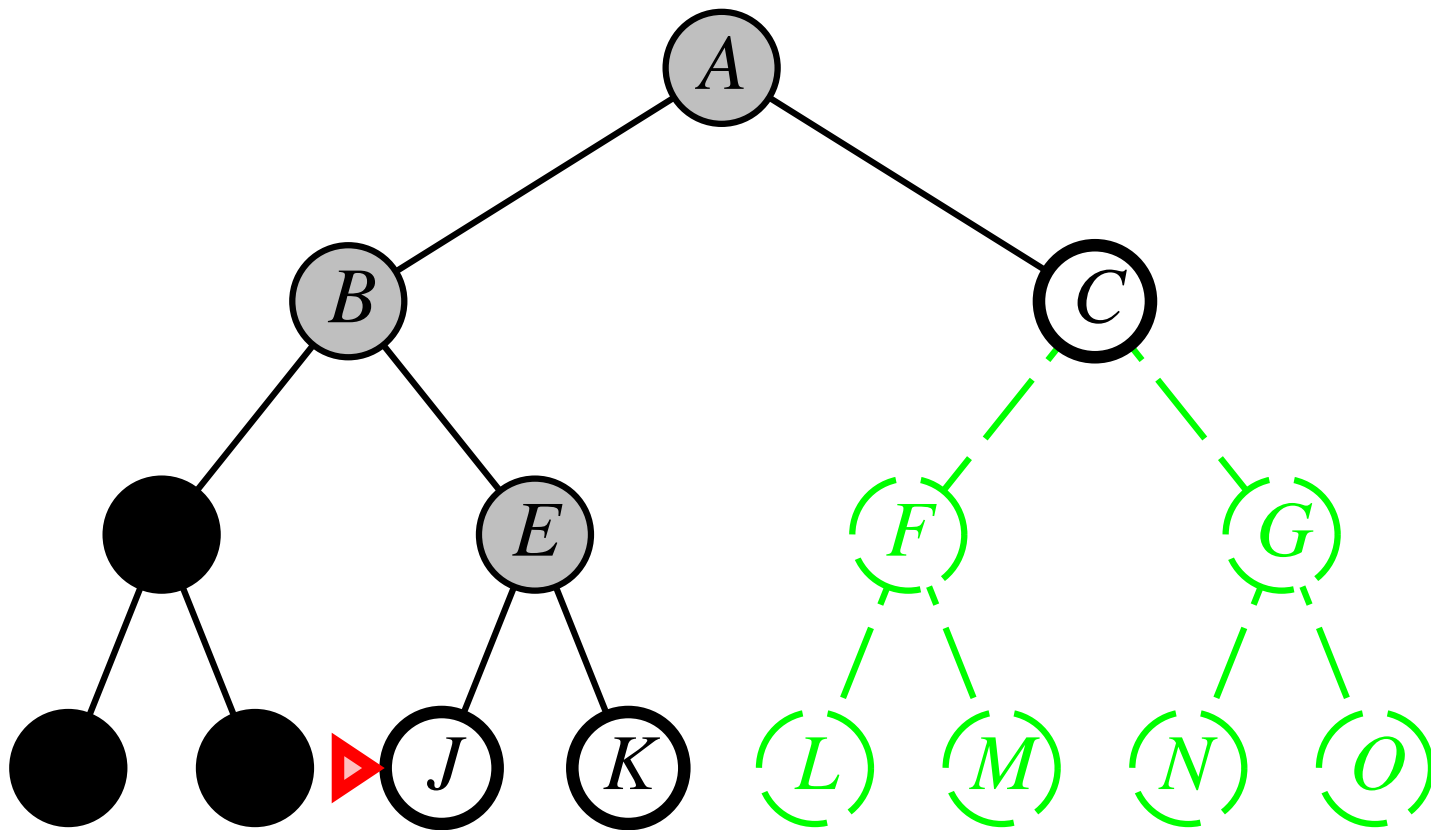
Depth-First Search

- Expand deepest unexpanded node
- Implementation:** *fringe* = LIFO queue, i.e., put successors at front



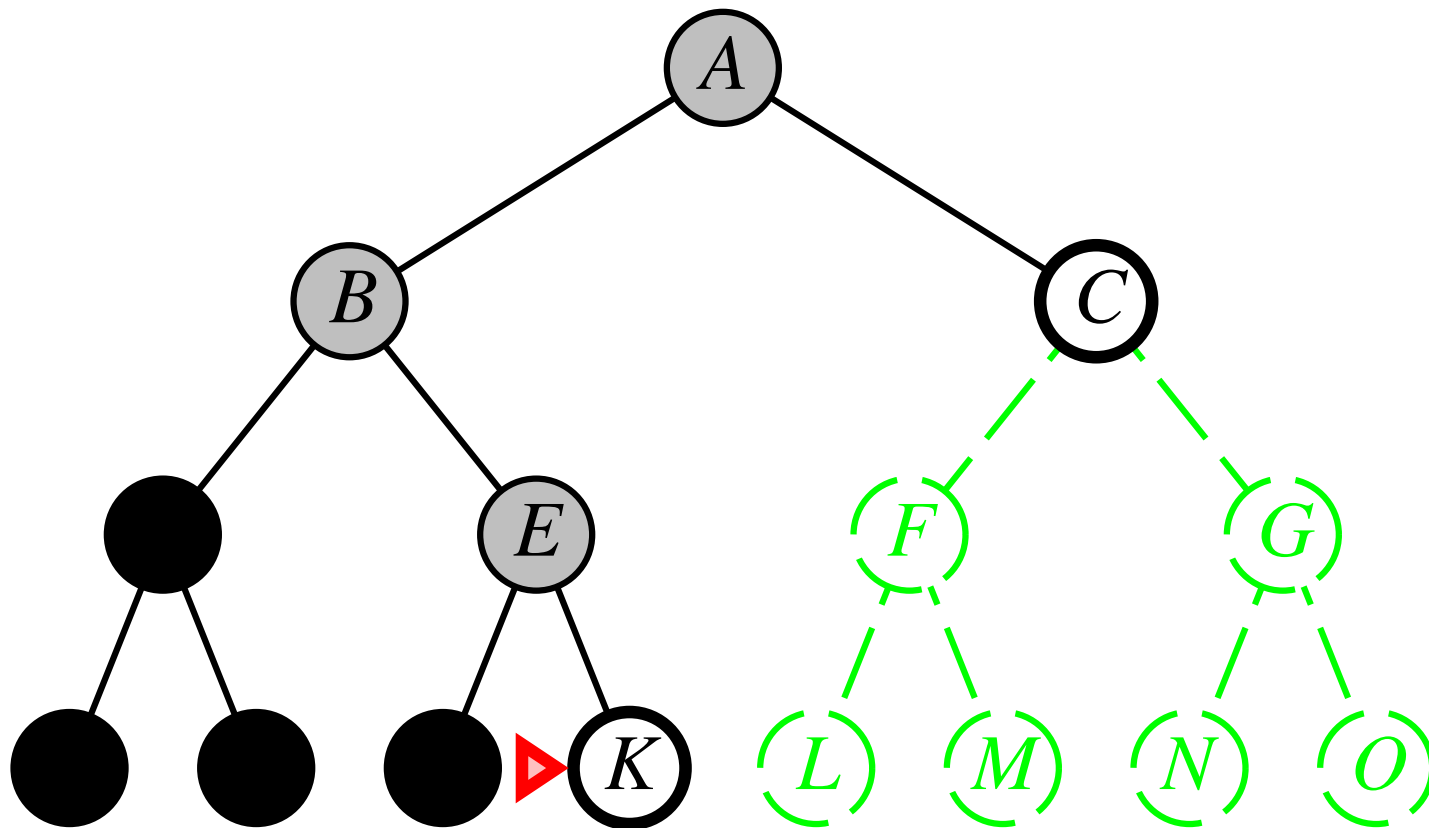
Depth-First Search

- Expand deepest unexpanded node
- Implementation:** *fringe* = LIFO queue, i.e., put successors at front



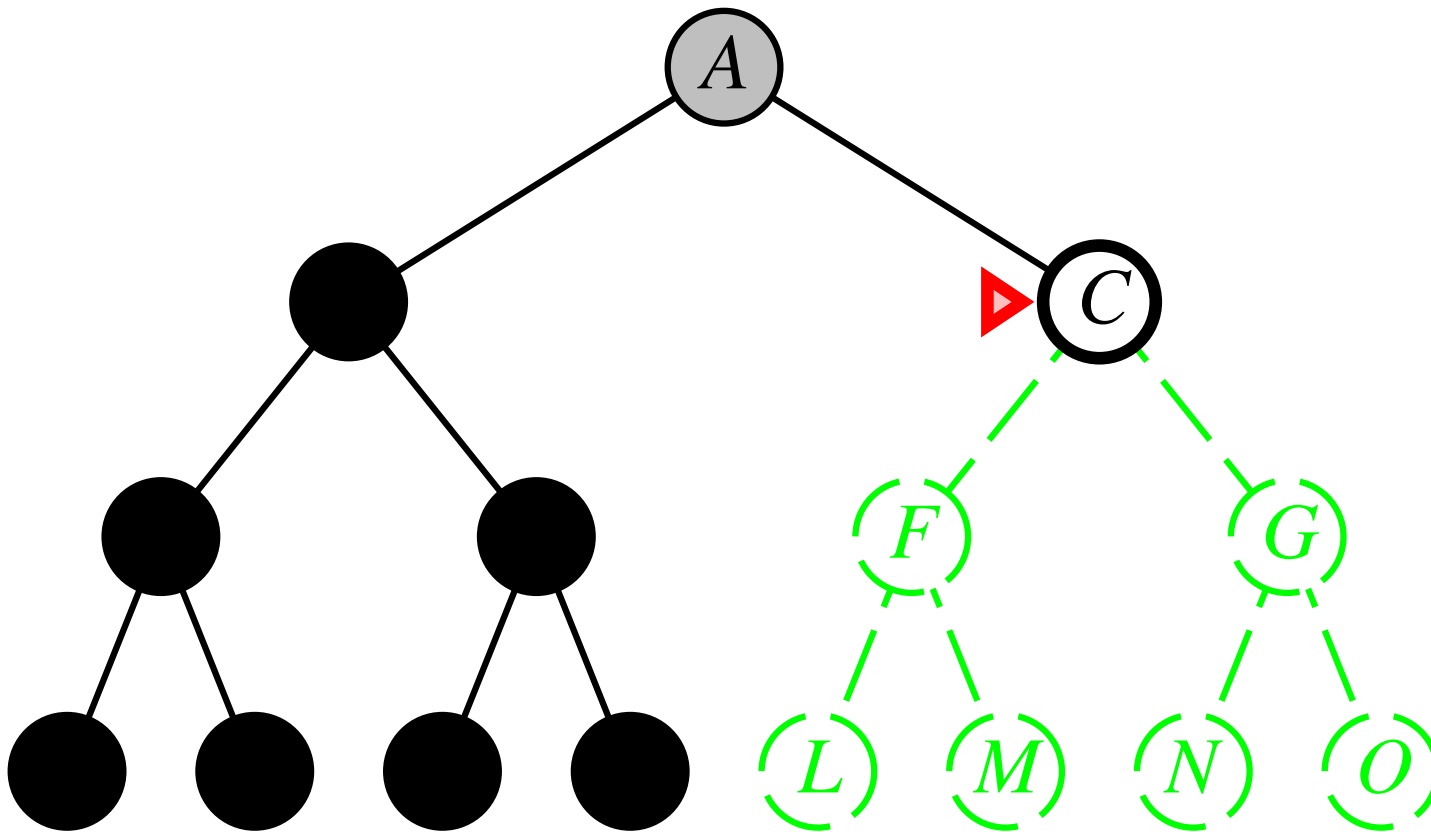
Depth-First Search

- Expand deepest unexpanded node
- Implementation:** *fringe* = LIFO queue, i.e., put successors at front



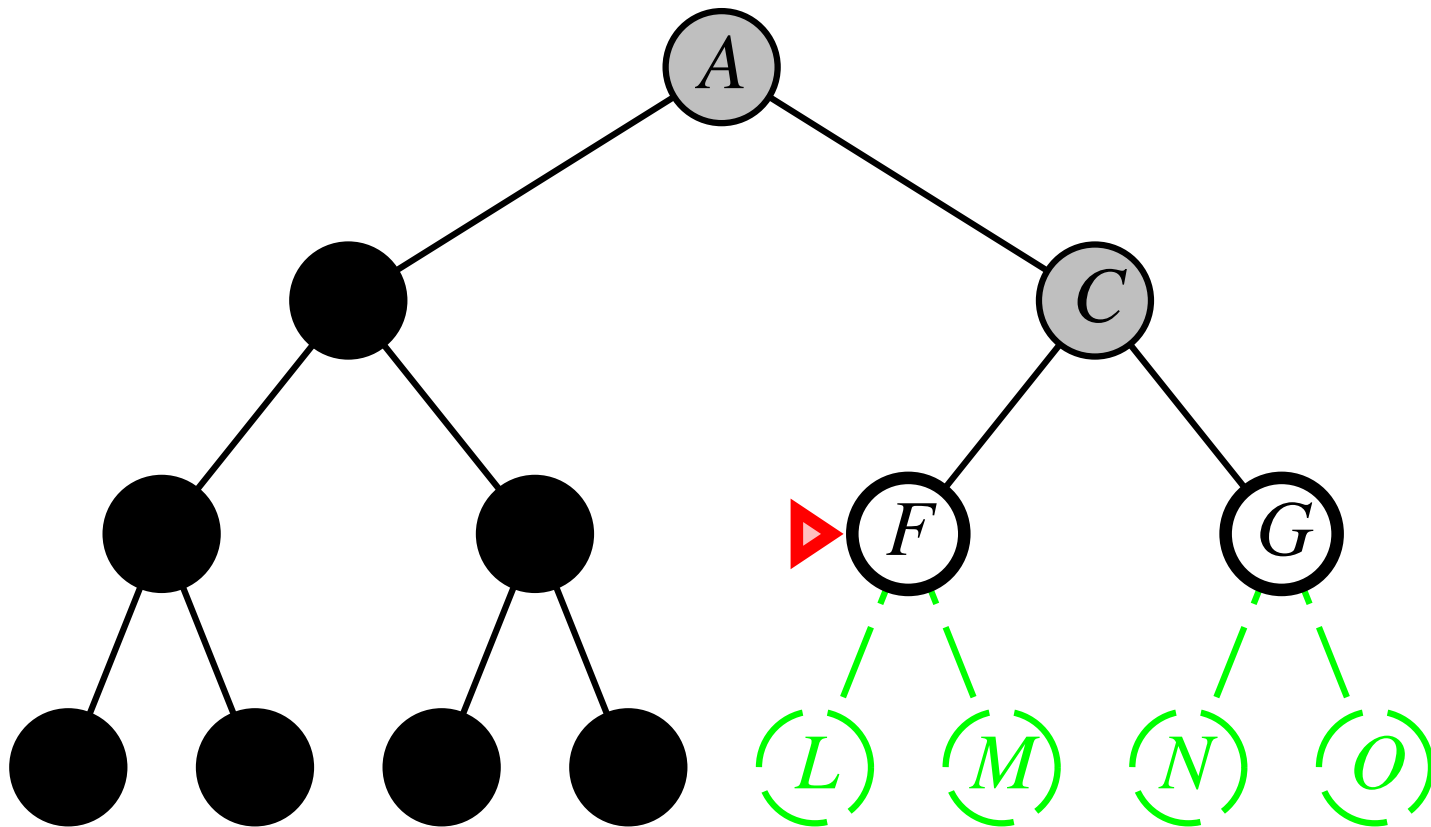
Depth-First Search

- Expand deepest unexpanded node
- Implementation:** *fringe* = LIFO queue, i.e., put successors at front



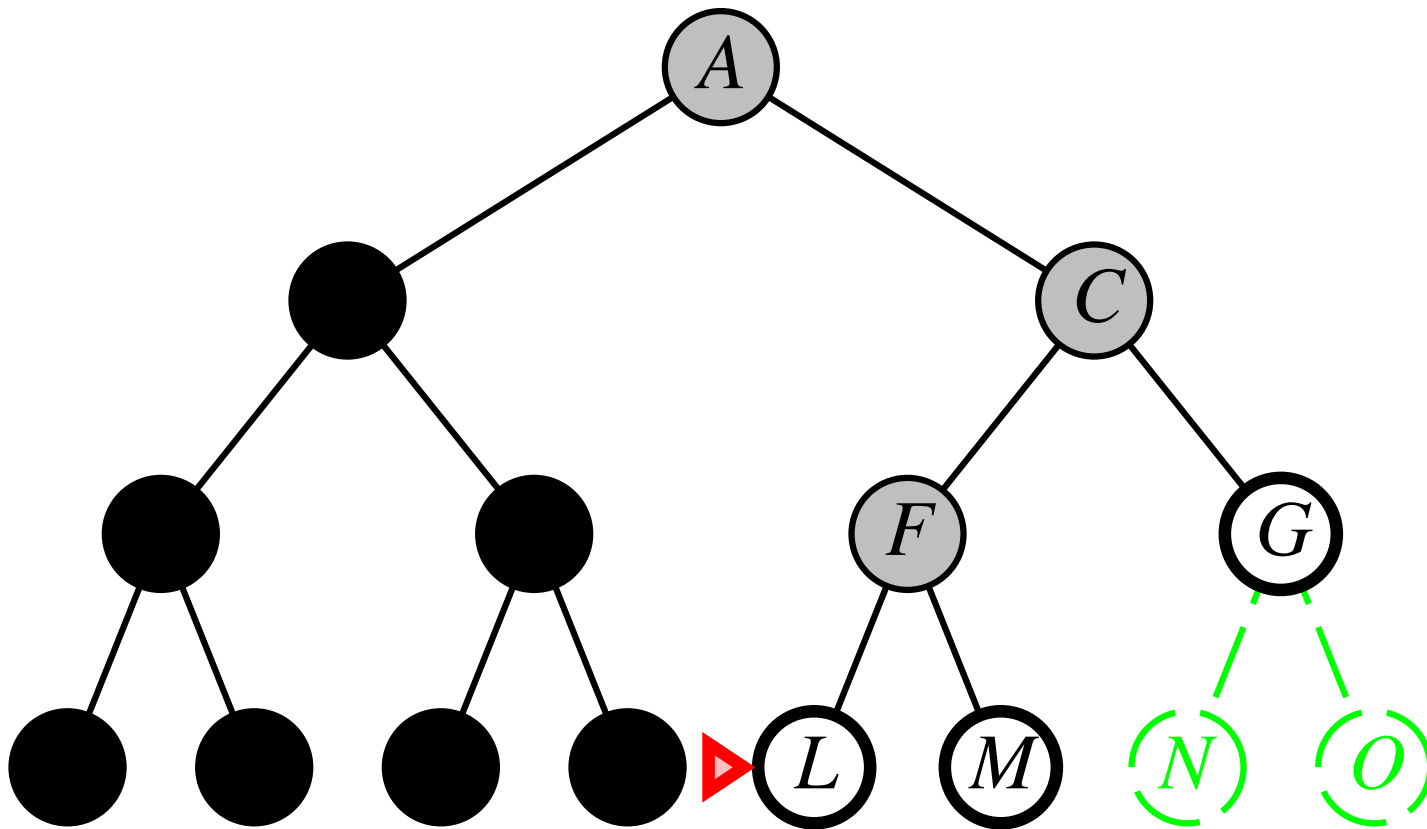
Depth-First Search

- Expand deepest unexpanded node
- Implementation:** *fringe* = LIFO queue, i.e., put successors at front



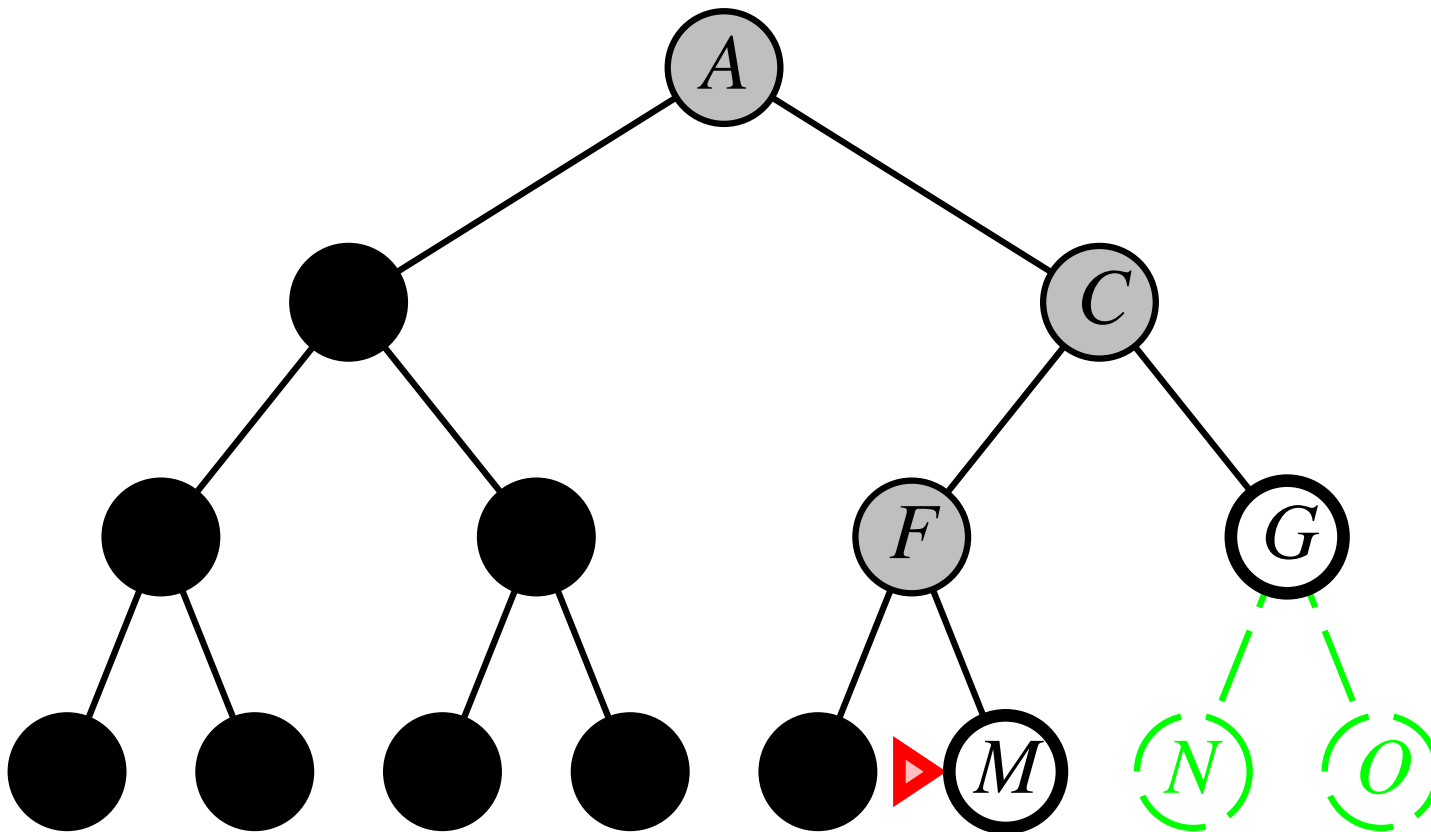
Depth-First Search

- Expand deepest unexpanded node
- Implementation:** *fringe* = LIFO queue, i.e., put successors at front

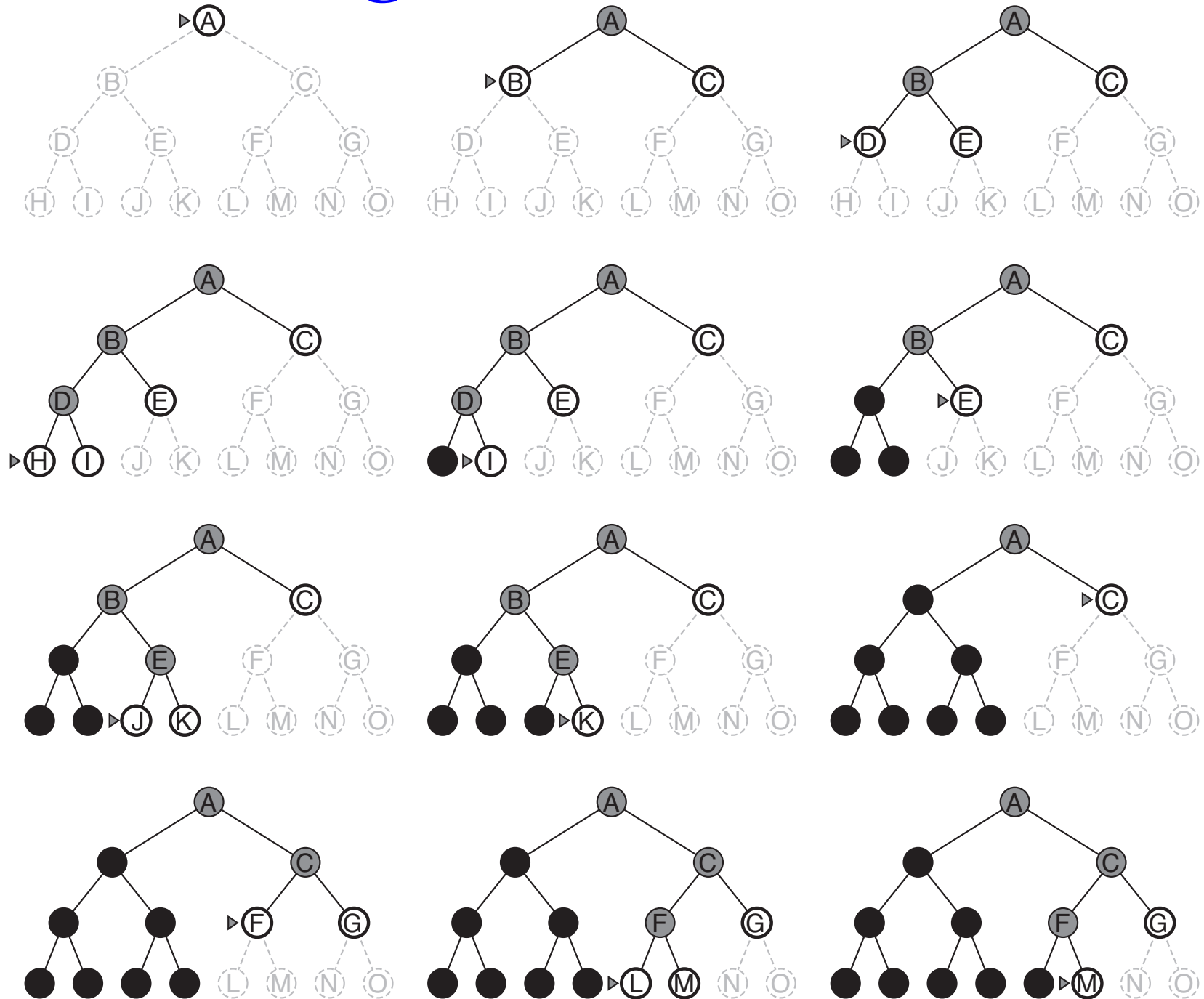


Depth-First Search

- Expand deepest unexpanded node
- Implementation:** *fringe* = LIFO queue, i.e., put successors at front



DFS Progress



Properties of Depth-First Search

● Complete??

Properties of Depth-First Search

- Complete?? No: fails in infinite-depth spaces, spaces with loops. Modify to avoid repeated states along path
⇒ complete in finite spaces
- Time??

Properties of Depth-First Search

- Complete?? No: fails in infinite-depth spaces, spaces with loops. Modify to avoid repeated states along path $\Rightarrow$ complete in finite spaces
- Time?? $O(b^m)$: terrible if m is much larger than d but if solutions are dense, may be much faster than breadth-first
- Space??

Properties of Depth-First Search

- Complete?? No: fails in infinite-depth spaces, spaces with loops. Modify to avoid repeated states along path $\Rightarrow$ complete in finite spaces
- Time?? $O(b^m)$: terrible if m is much larger than d but if solutions are dense, may be much faster than breadth-first
- Space?? $O(bm)$, i.e., linear space!
- Optimal??

Properties of Depth-First Search

- Complete?? No: fails in infinite-depth spaces, spaces with loops. Modify to avoid repeated states along path $\Rightarrow$ complete in finite spaces
- Time?? $O(b^m)$: terrible if m is much larger than d but if solutions are dense, may be much faster than breadth-first
- Space?? $O(bm)$, i.e., linear space!
- Optimal?? No

Depth-Limited Search

- = depth-first search with depth limit l ,
i.e., nodes at depth l have no successors
- Recursive implementation:

function DEPTH-LIMITED-SEARCH(*problem*, *limit*) **returns** soln/fail/cutoff
 RECURSIVE-DLS(MAKE-NODE(INITIAL-STATE[*problem*]), *problem*, *limit*)

function RECURSIVE-DLS(*node*, *problem*, *limit*) **returns** soln/fail/cutoff
 cutoff-occurred? $\leftarrow$ false
 if GOAL-TEST(*problem*, STATE[*node*]) **then return** *node*
 else if DEPTH[*node*] = *limit* **then return** *cutoff*
 else for each *successor* **in** EXPAND(*node*, *problem*) **do**
 result $\leftarrow$ RECURSIVE-DLS(*successor*, *problem*, *limit*)
 if *result* = *cutoff* **then** *cutoff-occurred?* $\leftarrow$ true
 else if *result* $\neq$ failure **then return** *result*
 if *cutoff-occurred?* **then return** *cutoff* **else return** failure

Iterative Deepening Search

```
function ITERATIVE-DEEPENING-SEARCH( problem) returns a solution
  inputs: problem, a problem

  for depth  $\leftarrow$  0 to  $\infty$  do
    result  $\leftarrow$  DEPTH-LIMITED-SEARCH( problem, depth)
    if result  $\neq$  cutoff then return result
  end
```

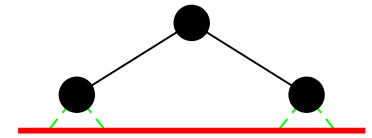
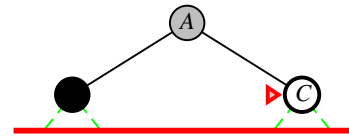
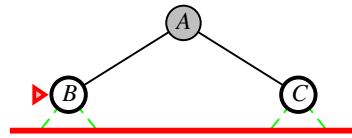
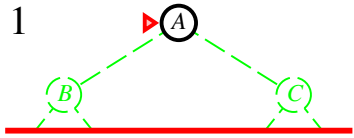
Iterative Deepening Search

it = 0



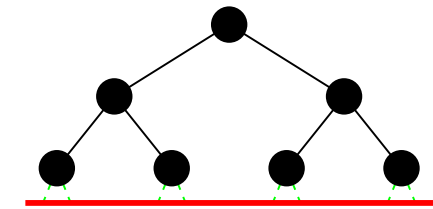
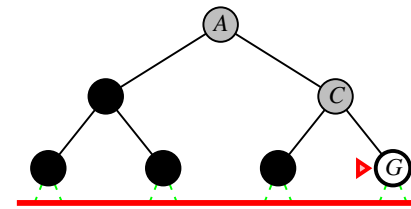
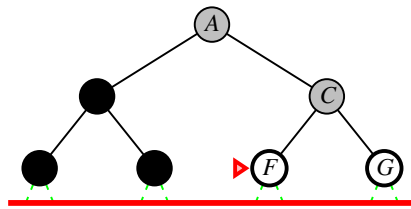
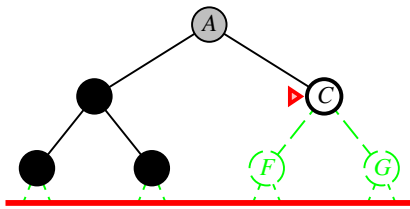
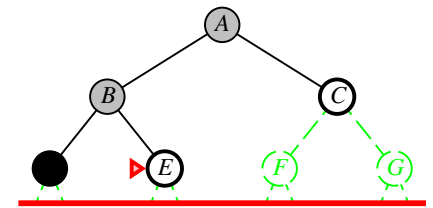
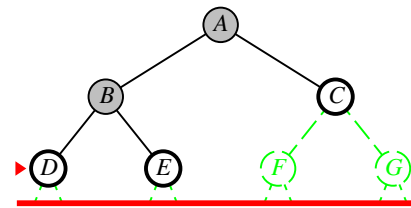
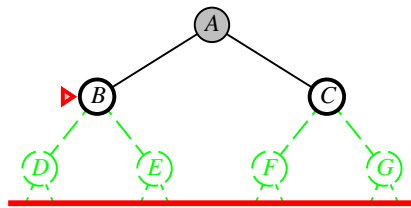
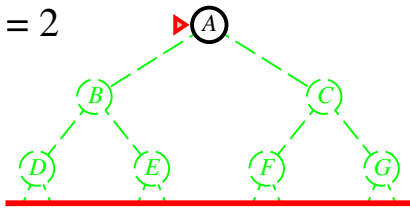
Iterative Deepening Search

nit = 1



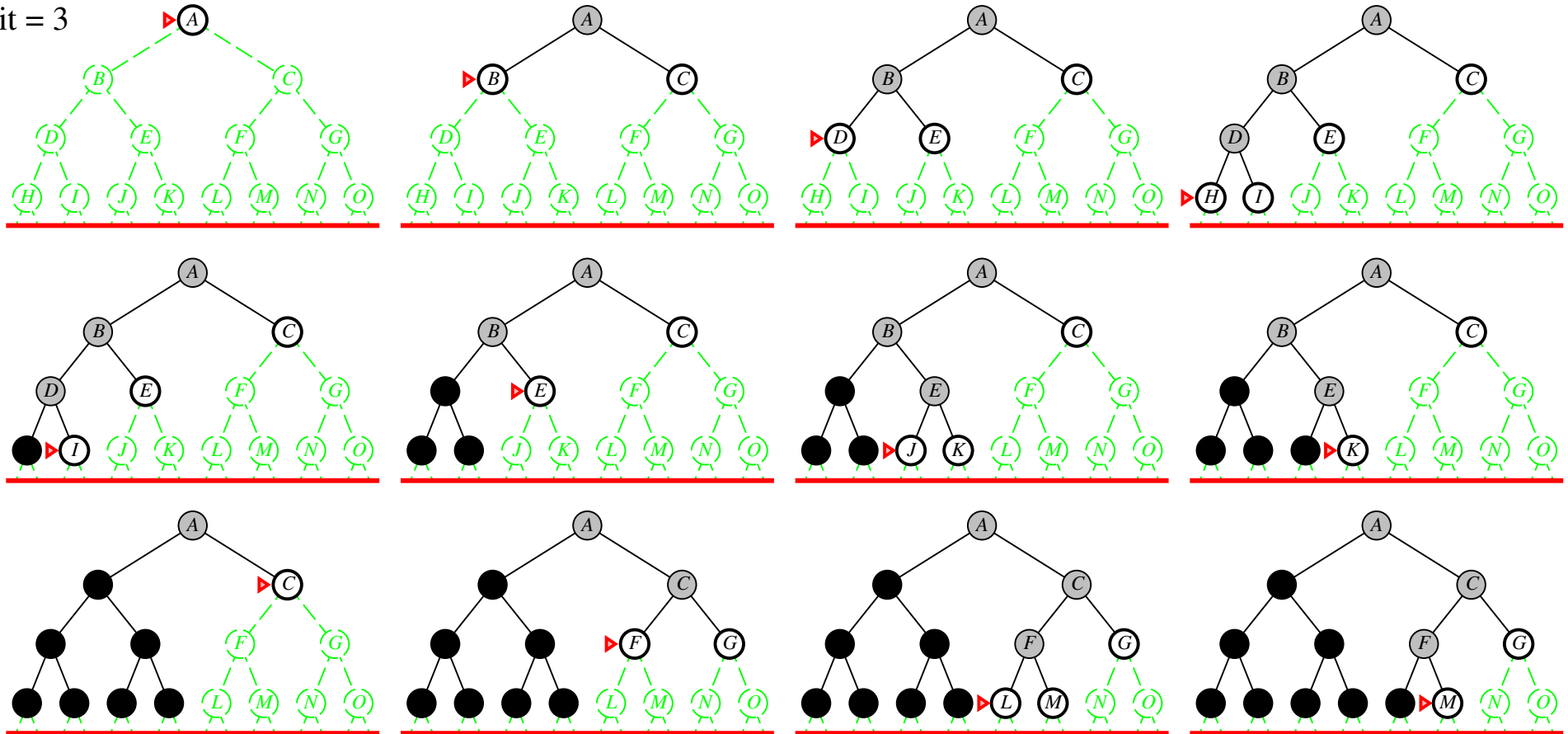
Iterative Deepening Search

limit = 2



Iterative Deepening Search

limit = 3



IDS Progress

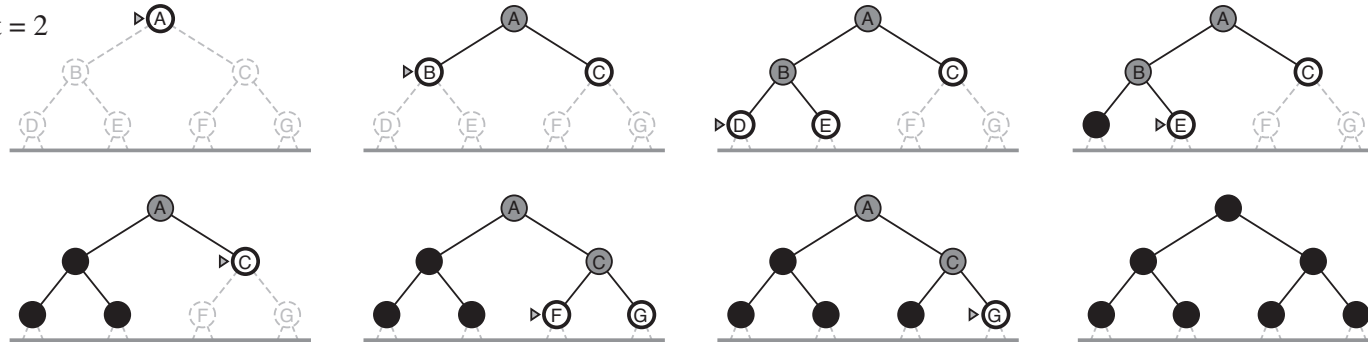
Limit = 0



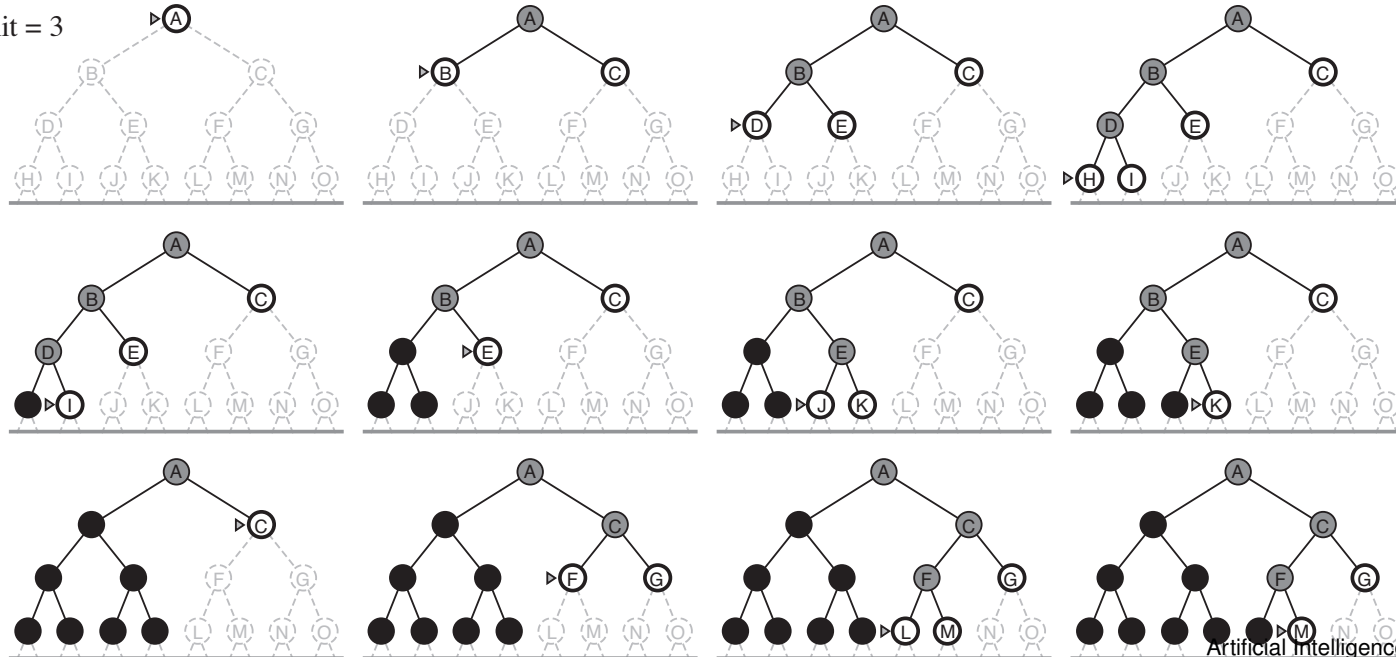
Limit = 1



Limit = 2



Limit = 3



Prop. of Iterative Deepening Search

● Complete??

Prop. of Iterative Deepening Search

- Complete?? Yes
- Time??

Prop. of Iterative Deepening Search

- Complete?? Yes
- Time?? $(d + 1)b^0 + db^1 + (d - 1)b^2 + \dots + b^d = O(b^d)$
- Space??

Prop. of Iterative Deepening Search

- Complete?? Yes
- Time?? $(d + 1)b^0 + db^1 + (d - 1)b^2 + \dots + b^d = O(b^d)$
- Space?? $O(bd)$
- Optimal??

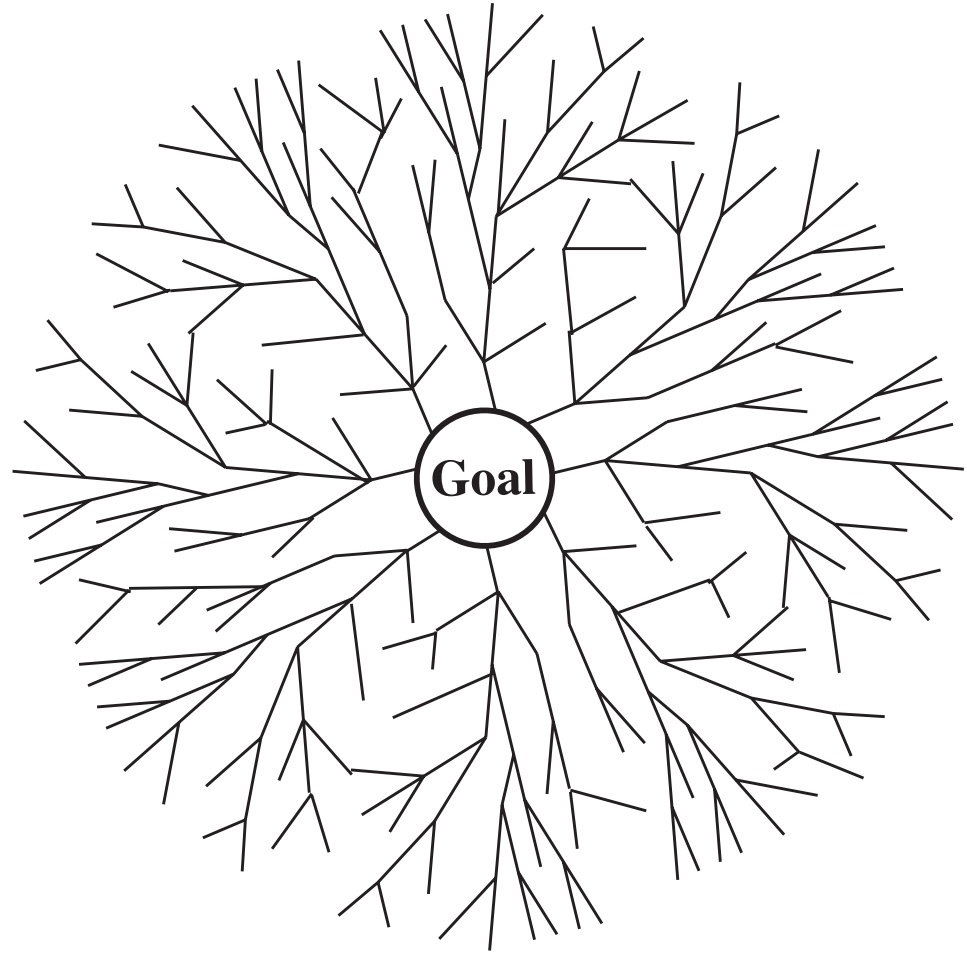
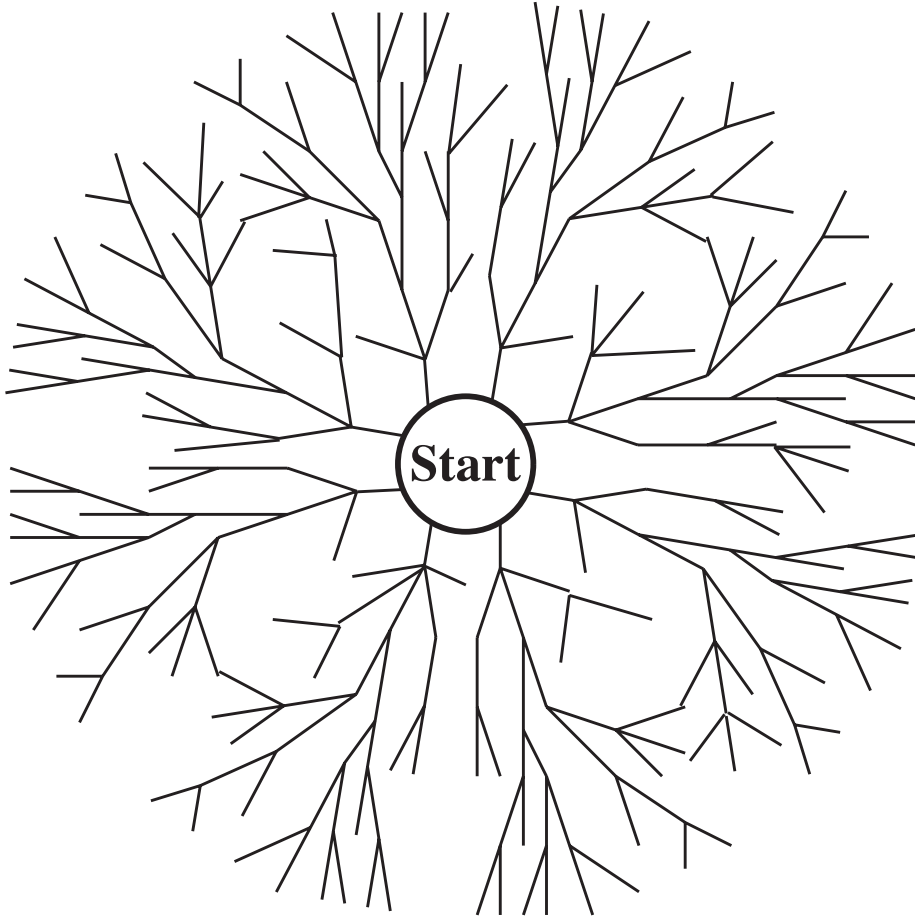
Prop. of Iterative Deepening Search

- Complete?? Yes
- Time?? $(d + 1)b^0 + db^1 + (d - 1)b^2 + \dots + b^d = O(b^d)$
- Space?? $O(bd)$
- Optimal?? Yes, if step cost = 1
Can be modified to explore uniform-cost tree
- Numerical comparison for $b = 10$ and $d = 5$, solution at far right:

$$N(\text{IDS}) = 50 + 400 + 3,000 + 20,000 + 100,000 = 123,450$$

$$N(\text{BFS}) = 10 + 100 + 1,000 + 10,000 + 100,000 + 999,990 = 1,111,100$$

Bidirectional

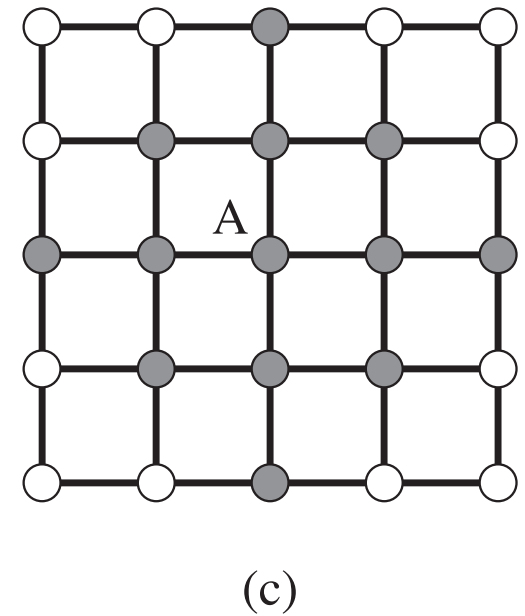
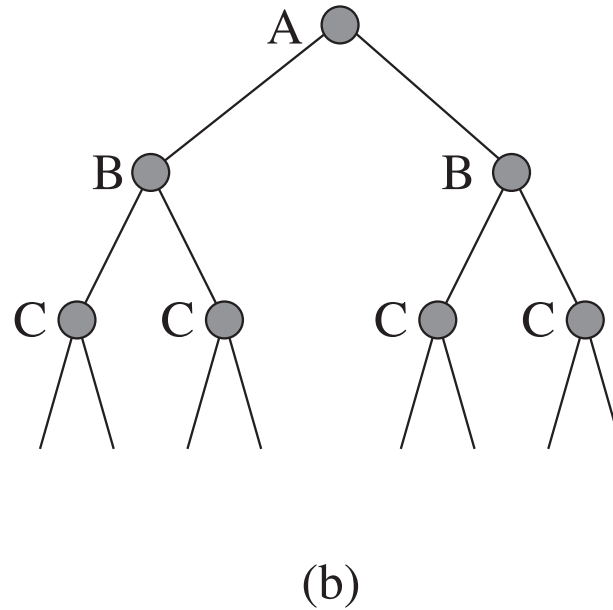
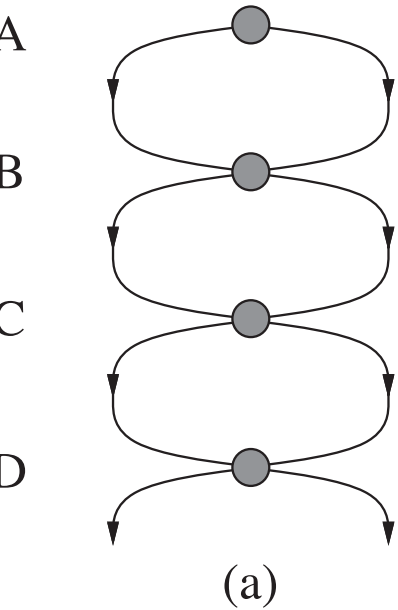


Summary of Algorithms

Criterion	Breadth- First	Uniform- Cost	Depth- First	Depth- Limited	Iterative Deepening
Complete?	Yes*	Yes*	No	Yes, if $l \geq d$	Yes
Time	b^{d+1}	$b^{\lceil C^*/\epsilon \rceil}$	b^m	b^l	b^d
Space	b^{d+1}	$b^{\lceil C^*/\epsilon \rceil}$	bm	bl	bd
Optimal?	Yes*	Yes	No	No	Yes*

Avoiding Repeated States

- Failure to detect repeated states can turn a linear problem into an exponential one!



Graph Search

function GRAPH-SEARCH(*problem*, *fringe*) **returns** a solution, or failure

closed $\leftarrow$ an empty set

fringe $\leftarrow$ INSERT(MAKE-NODE(INITIAL-STATE[*problem*]), *fringe*)

loop do

if *fringe* is empty **then return** failure

node $\leftarrow$ REMOVE-FRONT(*fringe*)

if GOAL-TEST(*problem*, STATE[*node*]) **then return** *node*

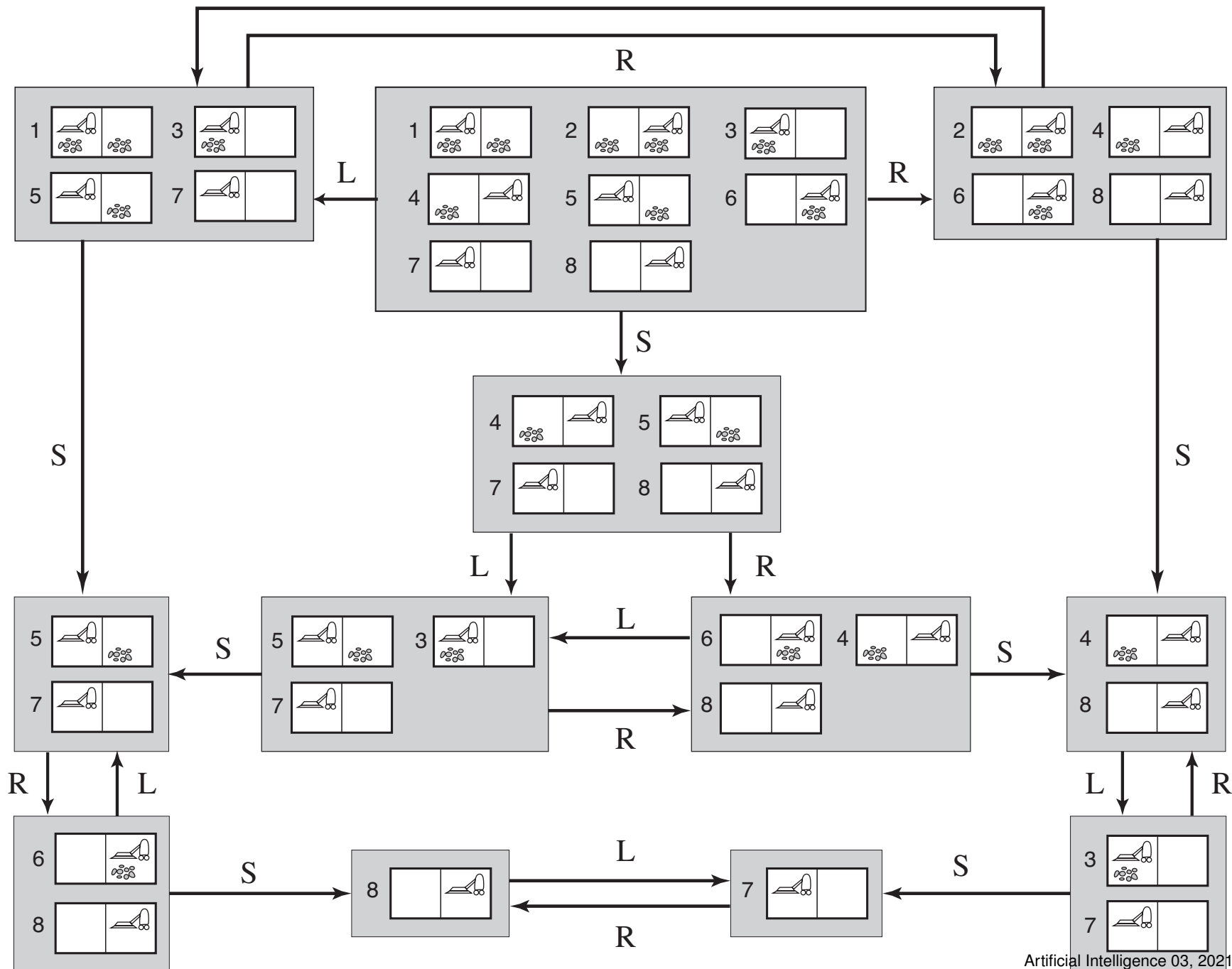
if STATE[*node*] is not in *closed* **then**

 add STATE[*node*] to *closed*

fringe $\leftarrow$ INSERTALL(EXPAND(*node*, *problem*), *fringe*)

end

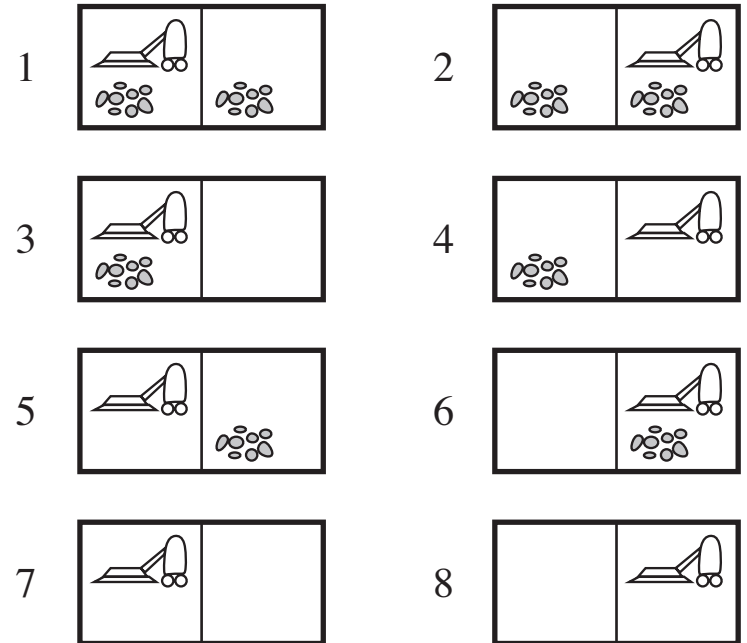
Sensorless (Conformant) Problems



Contingency Problems

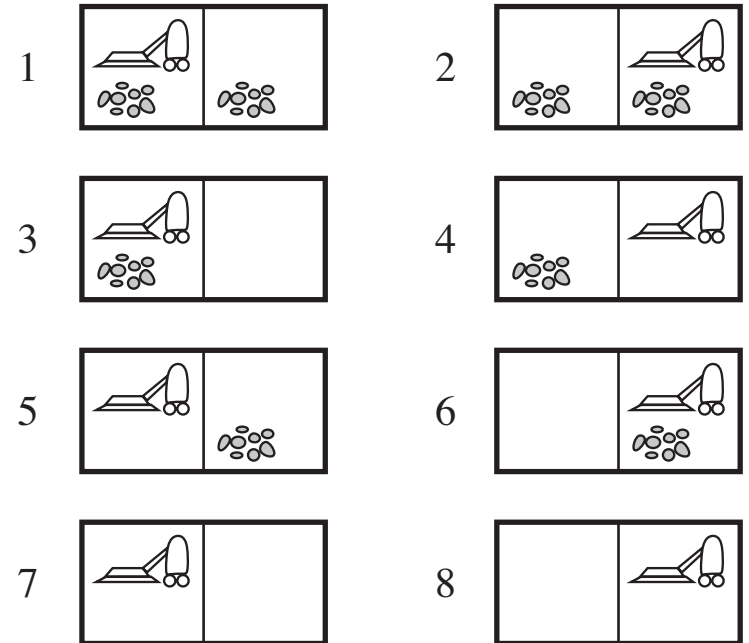
- Single-state, start in #5.

Solution??



Contingency Problems

- Single-state, start in #5.
Solution?? [*Right*, *Suck*]
- Conformant, start in
{1, 2, 3, 4, 5, 6, 7, 8}
e.g., *Right* goes to {2, 4, 6, 8}.
Solution??



Contingency Problems

- Single-state, start in #5.

Solution?? [*Right, Suck*]

- Conformant, start in

{1, 2, 3, 4, 5, 6, 7, 8}

e.g., *Right* goes to {2, 4, 6, 8}.

Solution??

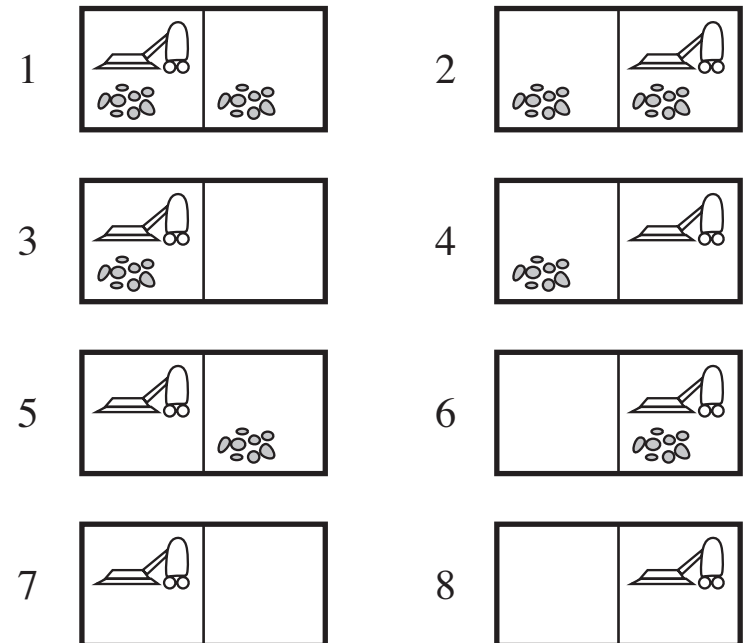
[*Right, Suck, Left, Suck*]

- Contingency, start in #5

Murphy's Law: *Suck* can dirty a clean carpet

Local sensing: dirt, location only.

Solution??



Contingency Problems

- Single-state, start in #5.

Solution?? [*Right, Suck*]

- Conformant, start in

{1, 2, 3, 4, 5, 6, 7, 8}

e.g., *Right* goes to {2, 4, 6, 8}.

Solution??

[*Right, Suck, Left, Suck*]

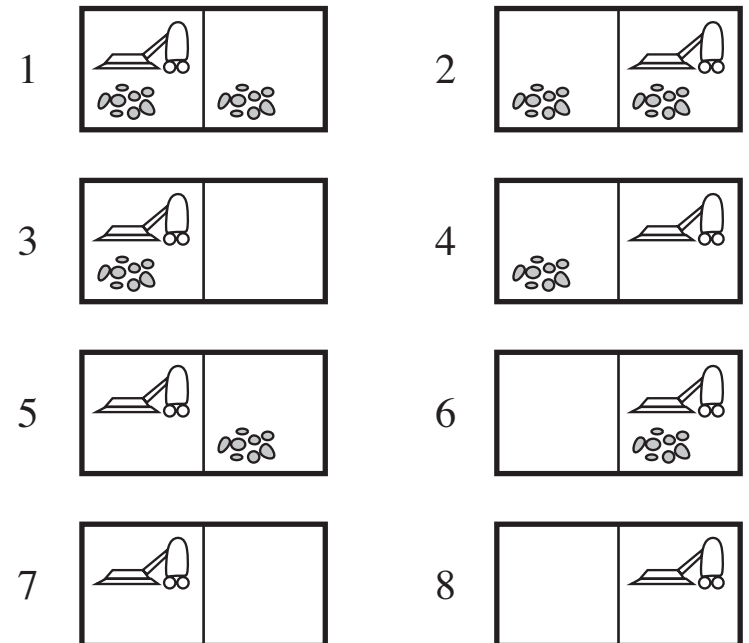
- Contingency, start in #5

Murphy's Law: *Suck* can dirty a clean carpet

Local sensing: dirt, location only.

Solution??

[*Right, if dirt then Suck*]



Summary

- Problem formulation usually requires abstracting away real-world details to define a state space that can feasibly be explored
- Variety of uninformed search strategies
- Iterative deepening search uses only linear space and not much more time than other uninformed algorithms

Exercise 3.8

Consider a state space where the start state is number 1 and the successor function for state n returns two states, numbers $2n$ and $2n + 1$.

- Draw the portion of the state space for states 1 to 15.
- Suppose the goal state is 11. List the order in which nodes will be visited for breadthfirst search, depth-limited search with limit 3, and iterative deepening search.
- Would bidirectional search be appropriate for this problem? If so, describe in detail how it would work.
- What is the branching factor in each direction of the bidirectional search?
- Does the answer to (c) suggest a reformulation of the problem that would allow you to solve the problem of getting from state 1 to a given goal state with almost no search?

Exercise 3.18

Consider the sensorless, two-location vacuum world under Murphy's Law. Draw the belief state space reachable from the initial belief state $\{1,2,3,4,5,6,7,8\}$, and explain why the problem is unsolvable. Show also that if the world is fully observable then there is a solution sequence for each possible initial state.